

PyTestGenie

Web Tool for Python Unit Test Code generation, Test Smell Detection
and Refactoring

Technical Report

SE 801: Software Project Lab III

Supervised by:

Toukir Ahammed

Lecturer

Institute of Information Technology

University of Dhaka

Submitted by:

Amit Kumar Roy

Roll: 1314

Institute of Information Technology

University of Dhaka

Submitted to:

SPL-3 Program Committee

Submission Date: February 26, 2026



Institute of Information Technology
University of Dhaka

Letter of Transmittal

February 25, 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission PyTestGenie (A tool for Automated Unit Test Generation , Smell Detection & Refactoring for Python)

Sir,

I am submitting the complete technical report for the Software Project Lab 3, "PyTestGenie: A web tool for Python Unit Test Code Generation , Test Smell Detection & Refactoring for Python"

This report covers the project's technical details, including design, implementation, methodology and testing plans. While I have tried to do my best and I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Amit Kumar Roy
BSSE-1314
Institute of Information Technology
University of Dhaka

Supervisor's Signature

Acknowledgment

I am very thankful to the Institute of Information Technology for granting me the opportunity to work on the Soft Project Lab III, PyTestGenie: A web tool for python unit test code generation, test smell detection and refactoring. I would like to express my gratitude to my supervisor, Toukir Ahammed (Lecturer at the Institute of Information Technology, University of Dhaka. His continuous support in working on this project, preparing the full technical report, and valuable feedback on my works help me to be confident and able to complete the project. As well as I would like to express my thanks to my classmates, seniors, their suggestions and support were very helpful throughout this process.

Abstract

This report includes requirements, usage scenarios, data based design, class diagrams, architectural design, ER diagram, implementation overview, test plans of PyTestGenie: A web tool for python unit test code generation, test smell detection and refactoring. We know every software project or system relies on unit tests to ensure that it remains correct and easy to maintain. Manually writing high quality unit tests requires much time and effort. Whereas automated unit test writing saves time. But there can be test smells which should be refactored.

PyTestGenie is designed to overcome these complexities. Here in a single application users can generate unit test code, detect smell and refactor. User can use both rule based approach and llm based approach. Users can use single AI agent or multi AI agent to refactor the code. This will help user to improve test quality, can understand how to refactor, where can be test smells. Besides this will also researcher to compare both rules based approach and llm based approach in python unit testing. Thus this reduce manual effort and time.

Table of Contents

1. Introduction.....	1
1.1 Motivation.....	1
1.2 Scope.....	1
1.3 Project Timeline.....	2
1.4 Purpose of Document.....	3
2. Project Description.....	3
2.1. Quality Function Deployment.....	3
2.1.1. Normal Requirements.....	3
2.1.2. Expected Requirements.....	4
2.1.3. Exciting Requirements.....	4
2.2. Usage Scenarios.....	4
2.2.1 Authentication.....	5
2.2.2 Test Code Generation.....	5
2.2.3 Test Smell Detection.....	5
2.2.4 Test Code Refactoring.....	5
2.2.5 User Dashboard.....	6
2.2.6 Project and Report Archive Management.....	6
3. Scenario-Based Modeling.....	6
3.1 Use Case Diagram.....	7
3.1.1 Level 0.....	7
3.1.2 Level-1.....	8
4. Database Modeling.....	10
4.1 ER Diagram and Schemas.....	11
5. Class-Based Modeling.....	13
5.1 Analysis Classes.....	13
5.2 Class cards.....	14
5.3 CRC Diagram.....	17
6. Architectural Design.....	18
6.1 Architectural Context Diagram.....	18
6.2 Defining Archetypes.....	19
6.2.1 Pipe and Filter Architecture.....	19
6.2.2 Layered Architecture Instantiation for PyTestGenie.....	20
7. Component Level Design.....	22
7.1 Top level Components.....	22
8. Deployment.....	26
9. Applied Methodology.....	26

9.1 Workflow.....	26
9.2 Input.....	27
9.3 Processing.....	28
9.4 Output.....	31
9.5 Code Implementation Overview.....	32
10. User Manual and User Interface.....	35
11. Test Plan.....	59
11.1 High-Level Description of Testing Goals.....	59
11.2 Test Cases.....	60
Conclusion.....	72
References.....	73

Figures

Figure-1: Timeline for PyTestGenie.....	2
Figure 2: Use Case Diagram Level 0.....	7
Figure 3: Use Case Diagram Level 1.....	8
Figure 4: ER diagram for user and user_dashboard.....	11
Figure 5: CRC Diagram.....	17
Figure 6: Architectural Context Diagram.....	19
Figure 7: Pipe and Filter Pattern for full testing process.....	19
Figure 8: Layered Architecture.....	20
Figure 9: Top Level Components.....	22
Figure 10: Deployment Diagram.....	26
Figure 11: High level workflow.....	27
Figure 12: Login Page.....	36
Figure 13: Registration form.....	37
Figure 14: “Pending approval” message shown after successful registration.....	38
Figure 15: Login page with credential entered.....	38
Figure 16: Application window after login.....	39
Figure 17: Dashboard.....	40
Figure 18: Change Password Window.....	41
Figure 19: Test Code Generator page.....	42
Figure 20: Rule based test case generation page.....	43
Figure 21: LLM based test case generation page.....	44
Figure 22: Rule based response for test code generation.....	45
Figure 23: LLM based response for test code generation.....	45
Figure 24: Detected smell viewer.....	46
Figure 25: Save version window.....	47
Figure 26: Button for report generation.....	47
Figure 27: Test Smell Detector page.....	48
Figure 28: LLM based test detector options.....	49
Figure 29: Smell detection results panel showing a table of detected smells.....	50
Figure 30 :LLM-based detection result with explanation text visible.....	50
Figure 31 :Refactoring panel after test smell detection.....	51
Figure 32 :Refactorer Page.....	52
Figure 33 :Test Smell Type Dropdown menu.....	53

Figure 34 :Refactored output page (Single Agent Mode).....	54
Figure 35 :Refactored output page (Multi Agent Mode).....	54
Figure 36 :My Projects page showing a list of projects.....	55
Figure 37 :Delete confirmation dialog.....	55
Figure 38 :Report Archive page showing a list of saved reports with titles and dates.....	56
Figure 39 :Report displayed showing all sections of the pipeline summary.....	56
Figure 40 :Admin Panel page showing the user management table.....	57
Figure 41 : A pending user state.....	58

Tables

Table-1: Schemas 11	
Table-2: CRC Card (User).....	14
Table-3: CRC Card (DashboardManager).....	14
Table-4: CRC Card (TestCodeGenerator).....	15
Table-5: CRC Card (TestSmellDetector).....	15
Table-6: CRC Card (RefactoringEngine).....	16
Table-7: CRC Card (ReportGenerator).....	16
Table-8: CRC Card (ArtifactsLibraryManager).....	16
Table-9: Matching requirements to software architecture.....	24

1. Introduction

This chapter gives a description of the PyTestGenie software requirements and specifications. It provides the purpose, scope, and target audience of the project, which is a clear basis on the implementation of the project.

1.1 Motivation

In modern times, it has been found that the correctness, reliability, and maintainability of Python applications have become more critical than ever. Unit testing is one of the programs of software quality but in reality it is a manual, time consuming and error prone exercise. The difficulty is also increased with Python because of dynamic typing, where automated generation and validation of unit tests is much more difficult than in the statically typed languages.

Even though Python has some tools to perform unit testing, the majority are concerned with executing tests and not the quality. To reduce the readability, reliability, and long-term maintainability, developers tend to create or write tests that have test smells and are poorly designed. Developers normally have to manually refactor even when these smells are recognized which adds to the time spent on development. In addition, the available solutions are usually dedicated to a particular IDE, are not automated, or use the latest developments in large language models (LLMs).

PyTestGenie will be used to minimize the work of developers, enhance the quality of tests. This is also open up to practitioners and researchers of advanced testing methods. Finally, this project aims at closing the gap between test generation, test quality analysis and automated refactoring.

1.2 Scope

The scope of the project:

- ☐ The system will ensure Python source code input in terms of either file upload or direct code pasting.
- ☐ It will create automated unit test cases based on rule-based and LLM.
- ☐ The system will identify usual Python test stencches like conditional test logic, assertion duplication and assertion roulette.
- ☐ It serves both rule based and llm based method to detect test smell.
- ☐ The framework will facilitate a role-based single and multi-agent (1-4 agents) workflow to refactor the code.
- ☐ Through an easy to understand interface, users will be able to see the results of analysis and detailed reports.

1.3 Project Timeline

TASKS	OCTOBER				NOVEMBER				DECEMBER				JANUARY				FEBRUARY			
	1st	2nd	3rd	4th	1st	2nd	3rd	4th	1st	2nd	3rd	4th	1st	2nd	3rd	4th	1st	2nd	3rd	4th
Prerequisite knowledge																				
Requirment Specification																				
Project Design																				
Development																				
Testing																				
Final Report Writing																				

Figure-1: Timeline for PyTestGenie

1.4 Purpose of Document

The purposes of this document are as follows:

1. Analyze the functional and non functional requirements of the project.
2. Define the architectural design of the automated unit test generation and smell detection pipeline.
3. Describe the rule-based and agentic LLM-based workflows used for test smell detection and refactoring.
4. Outline the methodology for test generation, smell detection, and automated refactoring.
5. Present the evaluation strategy for rule-based and LLM-based approaches.
6. Provide clarity on the project objectives, scope, and expected deliverables.

2. Project Description

PyTestGenie incorporates unit test generators, test smell detection and automated refactoring in one framework. The system eliminates the need to manually test and maximizes the maintainability of the tests by integrating rule-driven approaches with agentic, LLM workflows. PyTestGenie will be user friendly to both researchers and developers.

2.1. Quality Function Deployment

The main features of the PyTestGenie are the automation of the testing life-cycle, starting with the input of code and ending with the refactored unit tests.

2.1.1. Normal Requirements

The following are the key requirements that should be supported by the system:

1. The system will enable the user to upload Python source codes, paste or drop Python codes in a folder.
2. The system will scan and process the input code to be processed further.
3. The system will create unit test code based on rule-based techniques.

4. The system will identify test smells such as common test cases in generated or existing test cases.
5. The system on its part shall refactor the identified smells of the test to come up with clean and executable unit tests.

2.1.2. Expected Requirements

These attributes contribute to usability and effectiveness of the system and are not unreasonable to expect among the users:

1. The system must also be able to produce unit tests based on the LLM on top of rule-based techniques.
2. The system would aid role agentic workflow of test smell detection and refactoring.
3. The system must be able to explain test smells and refactorings implemented.
4. The system will have the analysis and refactoring results displayed to the user.

2.1.3. Exciting Requirements

These functionalities are interesting and provide a great deal of value:

1. Single-agent and multi-agent (role-based agent based on persona prompting) are integrated in the project.
2. The system can enable simultaneous comparison of different LLMs .
3. The system can be interactive and give feedback, allowing the user to understand the refactoring process.
4. The system is able to run full test suites in one integrated pipeline.

2.2. Usage Scenarios

PyTestGenien functions as a pipeline for test code generation, test smell detection and test code refactoring . The usage scenarios are as follows:

2.2.1 Authentication

Users can register by providing a username, email, and password. Account access is granted after admin verification. The system supports two roles: *Admin* and *User*, each with appropriate access permissions. User can also reset/forget his/her password with reset code which will be sent to his/her email.

2.2.2 Test Code Generation

PyTestGenie allows users to generate Python unit tests by providing source code through file upload or direct code input or can use file/folder from their user library. The system supports two complementary test generation approaches. In the first approach, rule-based automated test generation is used, where search-based algorithms systematically explore the program under test to generate unit tests that achieve high structural coverage where users can also select algorithms. In the second approach, PyTestGenie employs large language models (LLMs) to generate test cases by understanding program semantics, input constraints, and expected behavior.

2.2.3 Test Smell Detection

The system integrates test smell detection through the rule based (TEMPY). This is a well defined test smell detection system which is founded on the analysis of a given code using identified criteria. PyTestGenie is also developed with some test smell detection using LLM, which are used to detect the smells that might not be easily detected by rule-based detection techniques.

2.2.4 Test Code Refactoring

After test smell has been detected, PyTestGenie allows the test code to be refactored. The reason behind the refactoring process is mainly the techniques of LLM that make changes to the test code without altering its behavior. In case of complex the system uses role-based multi-agent LLM workflows, in which various agents cooperate to suggest, review and verify refactoring suggestions.

2.2.5 User Dashboard

Admin users are also able to see pending registration requests and approve or reject users, can user profile details. Also can Deactivate, activate user id. Users have options of controlling their profile settings such as changing their name and password.

2.2.6 Project and Report Archive Management

The project maintains an archive (user specific) that acts as storage for all user-related artifacts. This includes uploaded source code, generated unit tests, detected test smell reports, and refactored test code. This design supports effective management of testing artifacts without requiring repeated uploads or regeneration.

3. Scenario-Based Modeling

Scenario based modeling explains how a system behaves in different situations. These are based on user actions, events or certain conditions. In PyTestGenie scenario based modeling is used to describe how users interact with the system.

3.1 Use Case Diagram

A use case diagram illustrates how a system interacts with users, devices or other external entities. Generally these diagrams are created during the requirement analysis phase. By looking the diagrams everyone can easily understand the system's main functionalities.

3.1.1 Level 0

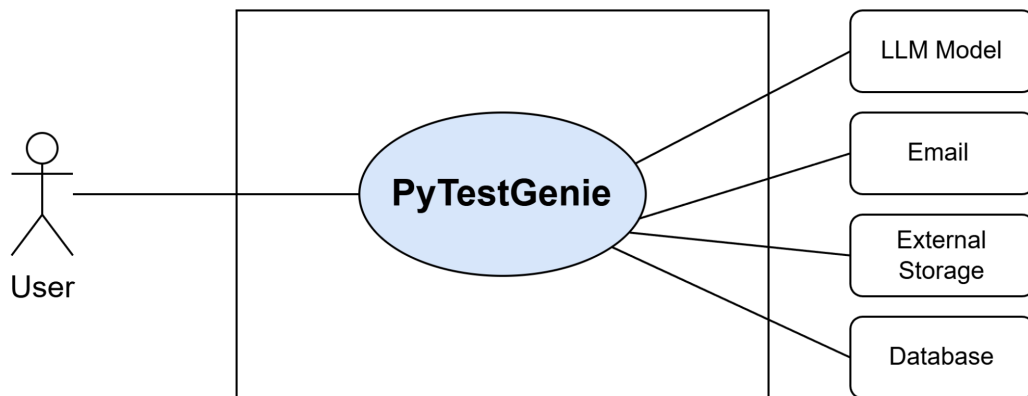


Figure 2: Use Case Diagram Level 0

Name: PyTestGenie

Primary actors: User

Secondary actors: LLM Model, Email Service, External Storage(e.g. MinIO), Database

Goal : The diagram represents the whole tool.

3.1.2 Level-1

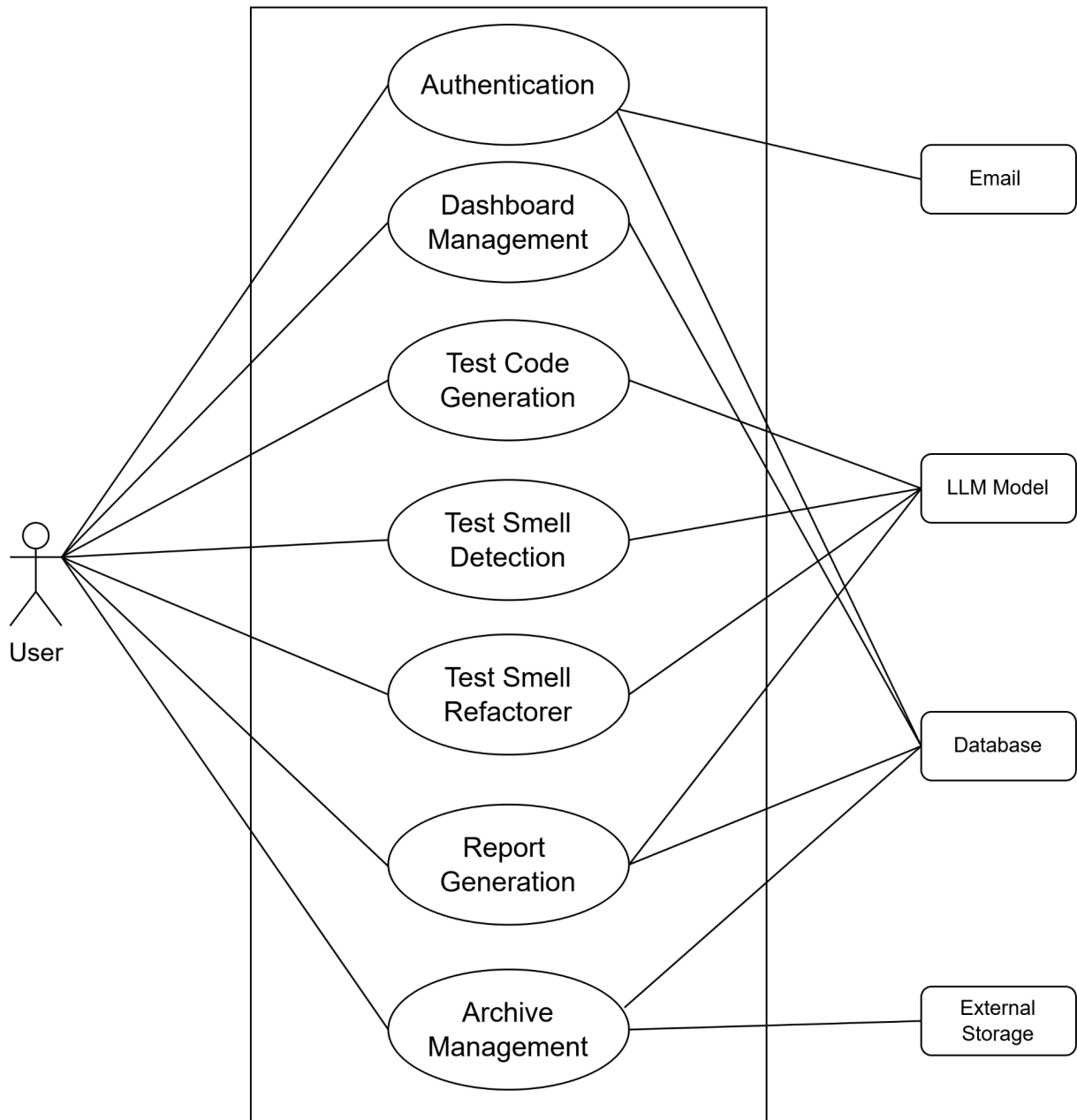


Figure 3: Use Case Diagram Level 1

Name: PyTestGenie

Primary actors: User

Secondary actors: LLM Model, Email Service, External Storage(e.g. MinIO), Database

Goal: The diagram shows the modules of PyTestGenie which are described below:

Authentication

This module handles user authentication (registration and login). New registered user can login after admin approval. User can also reset password using one time code which is sent to the registered email.

Dashboard Management

Admin users can view and manage pending registration requests, approve or reject users. Also update user profile (username, password), can see details statistics.

Test Code Generation

This module enables automated generation of Python unit tests from source code. PyTestGenie supports two approaches: Rule-based) test generation and LLM-based test generation, which leverages language models to understand program behavior and generate meaningful test cases. Users can also choose algorithms to use in rule-based approach and models to use LLM based approach.

Test Smell Detection

This module analyzes unit tests to identify test smells that negatively impact readability, maintainability, and reliability. PyTestGenie uses rule-based detection inspired by TEMPY, PyNOSE for well-defined smells, along with LLM-based analysis to identify more complex test smells. The detected smells are reported with clear explanations and code references.

Smell Refactorer

This module automatically refactors test code to eliminate detected test smells. It uses LLM-based refactoring techniques and supports role-based multi-agent workflows where users can choose (1-4) agents and different agents collaborate to propose, review, and validate refactoring suggestions.

Report Generation

This module creates detailed reports about every process (unit test code, detected test smell, refactored code). Users can view, download the html report. This will help anyone to understand.

Archive Management

The project maintains an archive (user specific) that acts as storage for all user-related artifacts. This includes uploaded source code, generated unit tests, detected test smell reports, and refactored test code. This design supports effective management of testing artifacts without requiring repeated uploads or regeneration.

4. Database Modeling

Database modeling describes how data will be stored in a system. That means it give clear picture of the database structure. For PyTestGenie, I have kept the database lightweight and focused on the core processing. So , analyzing the project, here I have kept 4 entities:

1. User
2. Projects
3. Project versions
4. Pipeline reports

4.1 ER Diagram and Schemas

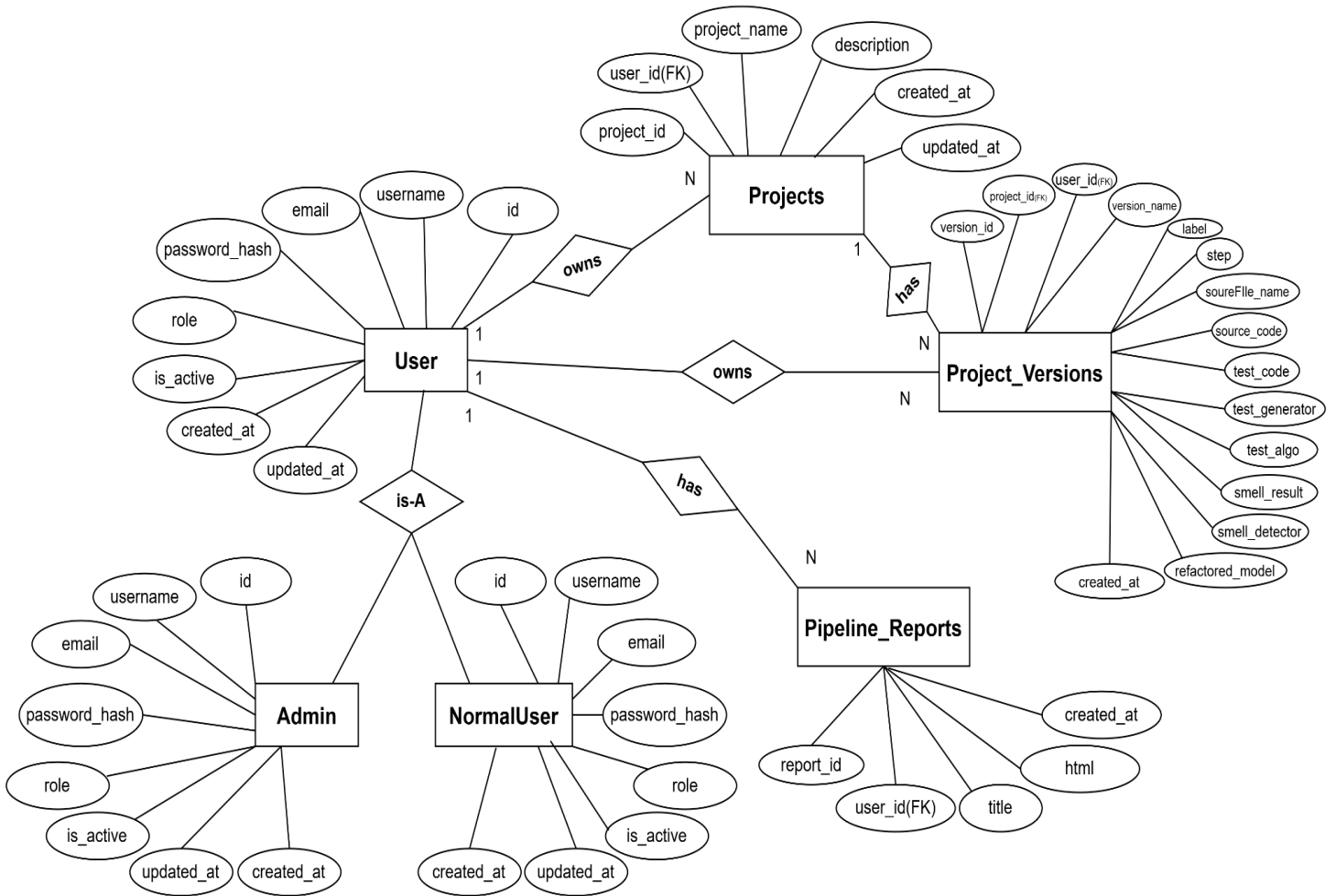


Figure 4: ER diagram for user and user_dashboard

The schemas of the entities are given below:

Table-1: Schemas

Schema	Attributes	Type
	id (PK)	INT
	username	VARCHAR(50)
	email	VARCHAR(50)

User (Admin/Normal User)	password_hash	VARCHAR(255)
	role	BOOLEAN
	is_active	BOOLEAN
	created_at	DATETIME
	updated_at	DATETIME
Projects	project_id (PK)	INT
	user_id (FK)	INT
	project_name	VARCHAR(50)
	description	TEXT
	created_at	DATETIME
	updated_at	DATETIME
Project_Versions	version_id(PK)	INT
	project_id(FK)	INT
	user_id(FK)	INT
	version_name	VARCHAR(50)
	label	VARCHAR(100)
	step	VARCHAR(100)
	sourceFile_name	VARCHAR(50)
	source_code	TEXT
	test_code	TEXT
	test_generator	VARCHAR(100)

	test_algo	VARCHAR(100)
	smell_result	TEXT
	smell_detector	VARCHAR(100)
	refactored_code	TEXT
	refactored_model	TEXT
	created_at	DATETIME
Pipeline_Reports	report_id(PK)	INT
	user_id(FK)	INT
	title	VARCHAR(50)
	html	LONGTEXT
	created_at	DATETIME

5. Class-Based Modeling

Class based modeling is way to describe the structure of the software system. It focuses classes, attributes and relationships with other entities. This is mainly object oriented design which helps developers to visualize the component of the system.

5.1 Analysis Classes

There are different selection criteria of retained information, services which are needed, attributes (common or individuals), common operations and essential requirement. Using this criteria and analyzing 56 potential nouns, the project can be described in 7 classes.

The selected 7 classes are:

1. **User**: Handles the secure authentication and role-based access for the platform.
2. **DashboardManager**: Manages user profiles.
3. **TestCodeGenerator**: Coordinates the creation of initial test suites using rule-based or LLM-based generation.
4. **TestSmellDetector**: Executes the dual-path detection (rule-based static analysis and agentic LLM analysis).
5. **RefactoringEngine**: Orchestrates the multi-agent workflow to transform smelly tests into high-quality code.
6. **ReportGenerator**: Aggregates data to generate reports.
7. **ArtifactLibraryManager**: save or delete file, code snippet, folder, generated reports.

5.2 Class cards

The class cards of PyTestGenie is given below:

Table-2: CRC Card (User)

User	
Attributes	Methods
- user_roles - user_id - credentials - is_active	+ authenticate_user() + authorize_access() + update_user()
Responsibilities	Collaborators
• Validates credentials and manages account registration. • Ensures only Admins can verify new user registrations.	DashboardManager, ArtifactsLibraryManager

Table-3: CRC Card (DashboardManager)

DashboardManager

Attributes	Methods
- user_name - user_role	+ load_dashboard() + update_profile()
Responsibilities	Collaborators
• Manage user profile	User

Table-4: CRC Card (TestCodeGenerator)

TestCodeGenerator	
Attributes	Methods
- Code_generation_method (LLM or Rule-based) - llm_model - test_generator_algorithm - code_context	+ generate_test_code() + display_test_code()
Responsibilities	Collaborators
• Generate unit tests using LLM or rule-based methods and present multiple versions for user inspection.	ReportGenerator ArtifactsLibraryManager

Table-5: CRC Card (TestSmellDetector)

TestSmellDetector	
Attributes	Methods
- detector_type (rule-based,LLM-based) - code_snippet - Smell_list	+ analyze_test_code() + detect_smells()
Responsibilities	Collaborators
• Identify test smells in generated test code using TEMPY rules or LLM analysis.	TestCodeGenerator, RefactoringEngine

Table-6: CRC Card (RefactoringEngine)

RefactoringEngine	
Attributes	Methods
- agents_list - refactored_code	+ detect_and_confirm_smells() + refactor_test_code() + validate_refactoring()
Responsibilities	Collaborators
• Coordinate multiple LLM agents to detect, confirm, and refactor test smells while ensuring correctness.	TestSmellDetector

Table-7: CRC Card (ReportGenerator)

ReportGenerator	
Attributes	Methods
- report_data - design_data - artifact(code/file's content)	+ generate_report() - calculate_metrics() - compile_results() + export_report()
Responsibilities	Collaborators
• Creates exportable reports for developers and researchers from test generation, smell detection, and refactoring.	TestCodeGenerator, TestSmellDetector, RefactoringEngine

Table-8: CRC Card (ArtifactsLibraryManager)

ArtifactsLibraryManager	
Attributes	Methods
- user_id(FK) - artifact_id(PK) - artifact_list - storage_path	+ save_artifact() + get_artifact_history() + delete_artifact()
Responsibilities	Collaborators

• Stores uploaded Python code and generated files (code, reports). • Provides the code to the Detection/Refactoring engines.	TestCodeGenerator, TestSmellDetector, RefactoringEngine
---	---

5.3 CRC Diagram

The CRC diagram of the classes can is given below:

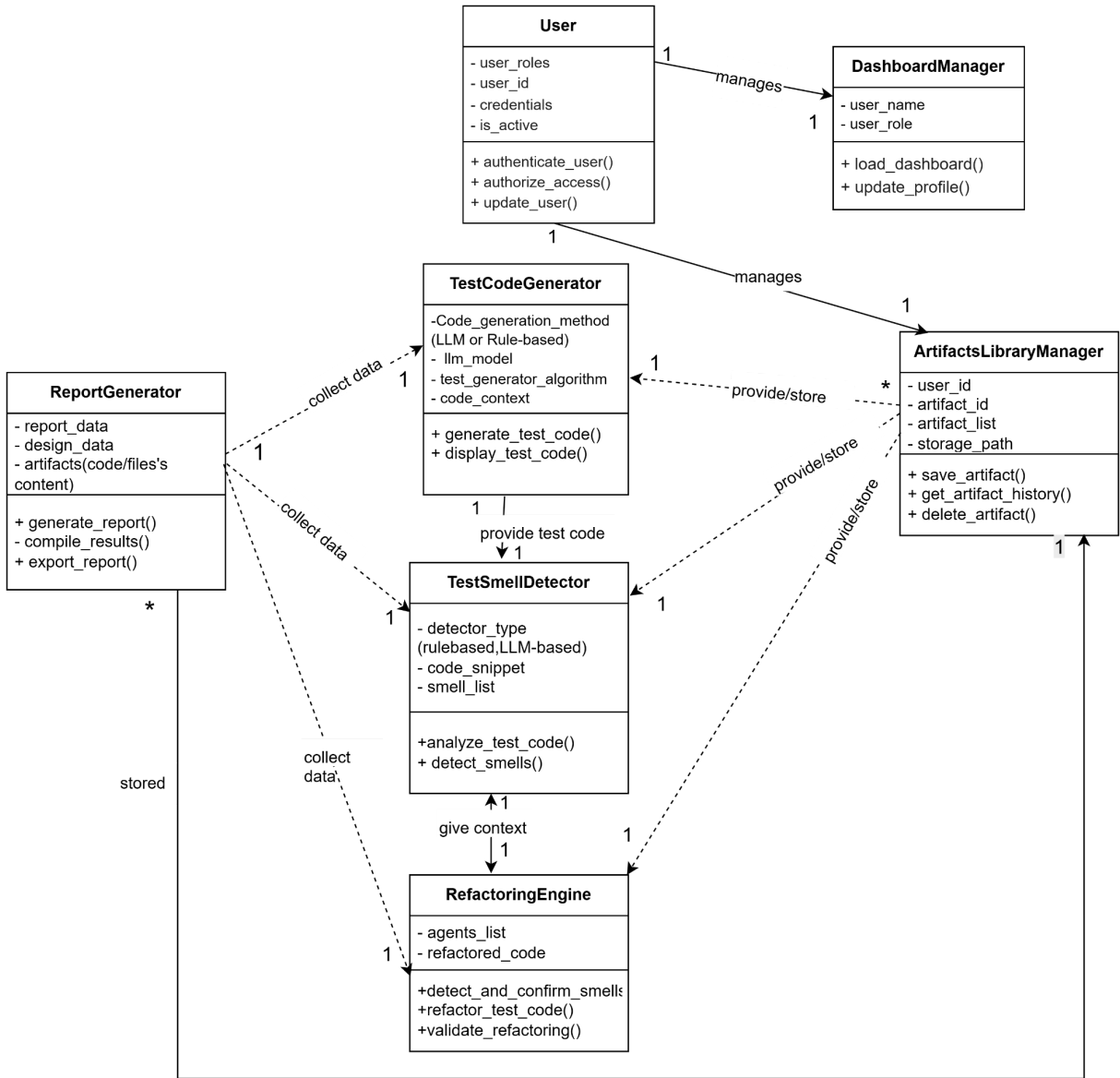


Figure 5: CRC Diagram

6. Architectural Design

Architectural design represents the high-level structure of a software system and how its major components are organized. It focuses on identifying key modules, defining their responsibilities, and describing how these components interact with each other. This design helps developers understand the overall system architecture, supports scalability and maintainability, and provides a foundation for detailed design and implementation decisions.

6.1 Architectural Context Diagram

At the architectural design level, an Architectural Context Diagram is used to illustrate how the target software system interacts with external entities beyond its system boundary. There are different categories:

- **Superordinate system:** The systems are larger.
- **Subordinate system:** The systems that the target system depends on. PyTestGenie relies on multiple subordinate systems to deliver its functionality. These include:
 - **LLM Services:** External large language model services are used for LLM-based unit test generation, intelligent test smell detection, and multi-agent-driven test code refactoring.
 - **External Storage:** An object storage system (e.g., MinIO) is used to store uploaded source code, generated test cases, refactored test code, and analysis reports within each user's artifact library.
 - **Database System:** A database is used to manage user authentication, role-based access control, artifact metadata, test smell records, and system configuration information.
 - **Email Service:** An email service is used for account verification, admin approval notifications
- **Peer-level system:** These systems exchange information/data. PyTestGenie does not interact with any peer-level systems.
- **Actors:** Developers, researchers, students, teachers are the primary users of PyTestGenie.

The architectural context diagram for the tool is given below-

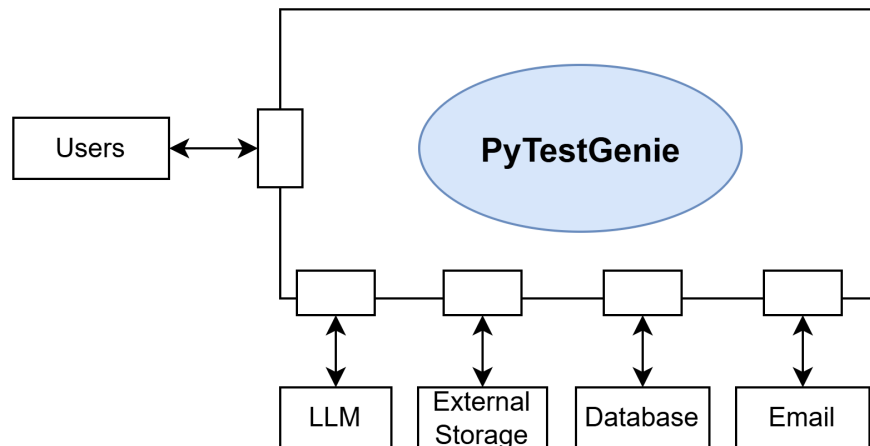


Figure 6: Architectural Context Diagram

6.2 Defining Archetypes

Our system employs pipe and filter, layered architectural patterns that work together to create a pipeline.

6.2.1 Pipe and Filter Architecture

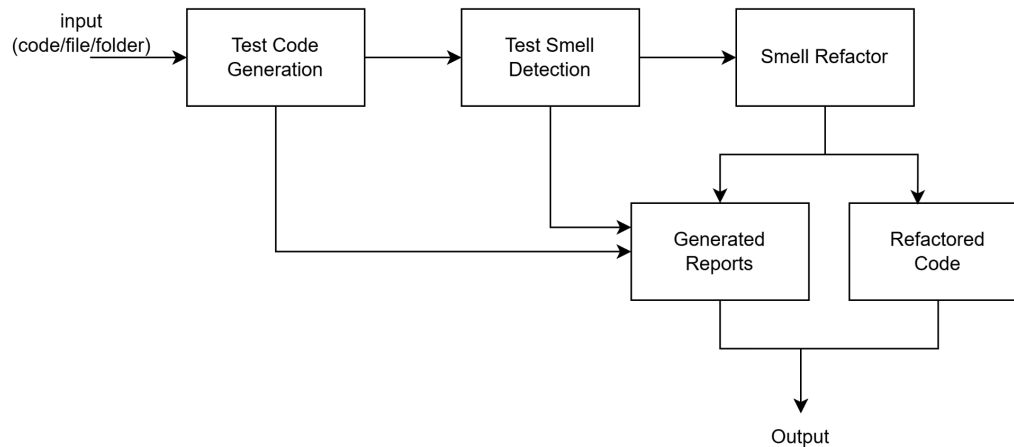


Figure 7: Pipe and Filter Pattern for full testing process

PyTestGenie employs a Pipe and Filter architectural pattern as the core design for its automated testing pipeline. This pattern is ideal for systems that require sequential data

transformation through multiple processing stages, where each stage (filter) performs a specific operation and passes the result to the next stage through connectors (pipes).

6.2.2 Layered Architecture Instantiation for PyTestGenie

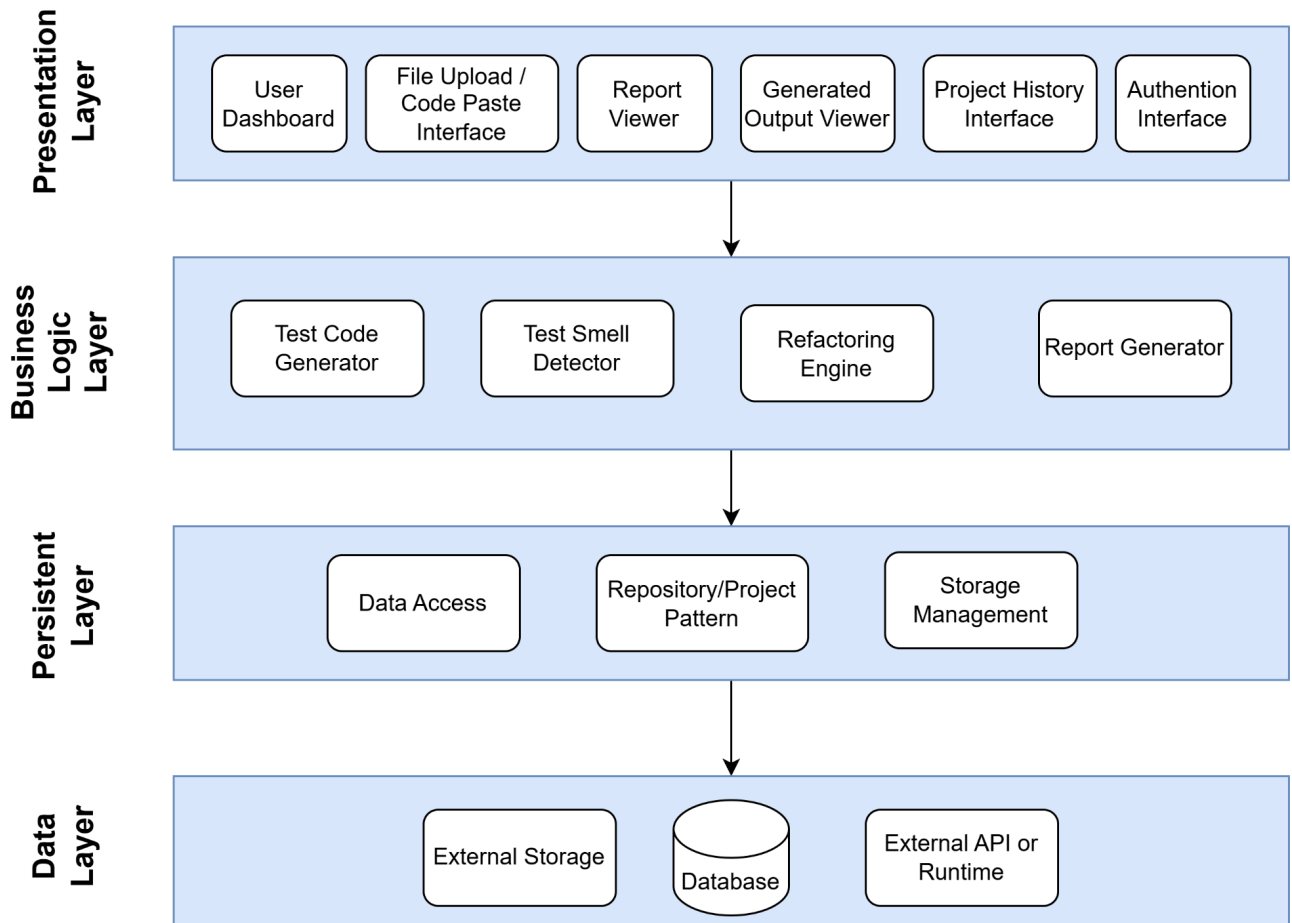


Figure 8: Layered Architecture

1. Data Layer

Components: Database, External Storage, External API or Runtime

Responsibilities:

- Stores all user's data/info, artifacts, including source code, generated tests, detected test smells, refactored tests, and reports.
- Integrate with and retrieve data from external services such as Hugging Face models, Ollama, and Google Gemini.

2. Persistent Layer

Components: Data Access, Project/Repository Pattern, Storage Management

Responsibilities:

- Provides CRUD operations for the artifacts and user data.
- Map domain objects to database entities.

3. Business Logic Layer

Components: TestCodeGenerator, TestSmellDetector, RefactoringEngine, ReportGenerator

Responsibilities:

- Coordinates test generation, smell detection, and automated refactoring workflows.
- Produced reports show test code, detected smells, and refactoring results.
- Manages role-based multi-agent workflows for enhanced accuracy and validation.

4. Presentation Layer

Components: User dashboard, File upload / Code paste interface, Report viewer, Authentication interface, Generated output viewer, Project history interface

Responsibilities:

- Provides user-friendly interfaces for authentication, uploading code, generating tests, detecting smells, refactoring, and viewing reports.
- Displays summary statistics and artifact usage per user.
- Present generated artifacts (tests, refactored codes, and analysis results) in a clear and structured format.

7. Component Level Design

7.1 Top level Components

The top-level components of the system are illustrated below. These components provide a high-level overview of the system's structure and functionality. Further refinement and detailed design are carried out in subsequent stages to better define the internal behavior and interactions of each component.

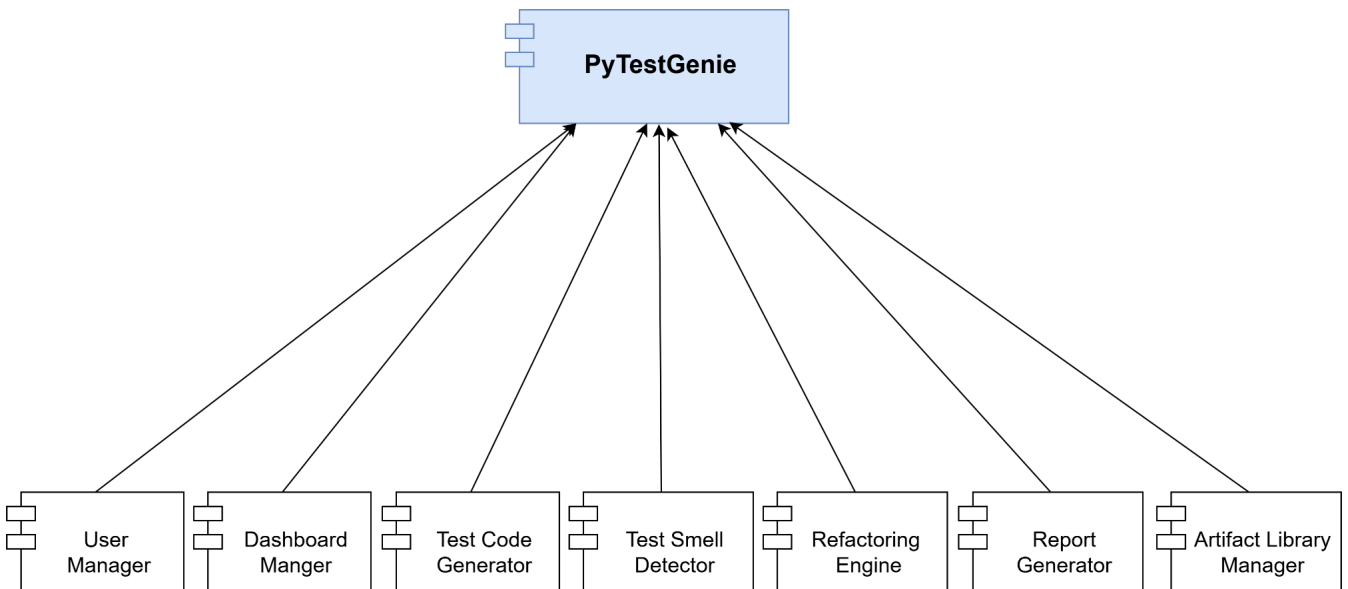


Figure 9: Top Level Components

Component 1: User Manager

This component acts as the security gateway of the system. It handles user registration (creating accounts in a pending state), login authentication using bcrypt password verification, and issues JWT tokens (HS256, 30-minute expiry). It enforces a two-tier access model where admin approval is required before regular users gain access. It manages password reset via one-time email tokens sent through SMTP and validates them upon submission.

Component 2: Dashboard Manager

This component displays user-specific statistics and activity summaries. It shows metrics related to test generation, detected test smells, refactored code, recently worked files, and overall usage. It also provides profile management features such as displaying the username and allowing password updates. Additionally, it presents aggregate system statistics collected from project history and pipeline activities.

Component 3: Test Code Generator

This component automatically generates unit tests for Python source files using either rule-based or AI-based strategies. Users can upload or paste source code. In AI mode, it resolves project dependencies, constructs context-aware prompts, and queries LLMs (such as GPT-OSS 20B or Llama 3.2) to generate pytest-compatible test suites, validating output via AST parsing. In rule-based mode, it invokes algorithms (DYNAMOSA, MIO, MOSA) and captures the generated tests. Pynguin logs are streamed in real time using Server-Sent Events (SSE). Generated tests are forwarded for smell detection, reporting, and versioned storage.

Component 4: Test Smell Detector

This component analyzes test code to detect structural and behavioral test smells. In rule-based mode, it uses Python AST parsing and the TEMPY static analysis engine to detect predefined smell categories. In LLM-based mode, it sends structured prompts containing smell definitions to Ollama or HuggingFace models to detect additional smell types. It can optionally enhance results with natural-language explanations generated by the Gemini API, with static fallback descriptions if needed. Detection outputs are forwarded for refactoring, reporting, and versioned storage.

Component 5: Refactoring Engine

This component refactors smelly test code into clean, standards compliant code using LLM-based strategies. In single-agent mode, it performs one-shot refactoring using targeted prompts. In multi-agent mode, it follows a four-stage pipeline: detection validation, evaluator review, refactoring generation, and final validation against smell removal, coding standards, and syntax correctness. It supports multiple smell types and

several LLM backends via Ollama and HuggingFace. Refactored outputs are included in reports and saved as versioned artifacts.

Component 6: Report Generator

This component generates and manages comprehensive pipeline reports. It aggregates results from test generation, smell detection, and refactoring into a styled HTML report containing summaries, per-file breakdowns, and detailed smell descriptions. Reports are archived in MongoDB with metadata and timestamps. Users can list, preview, delete, and download reports as HTML files.

Component 7: Artifact Library Management

This component provides persistent, versioned storage for all pipeline artifacts. Users can create named projects and save complete snapshots at any stage, including source code, generated tests, detection results, refactored code, and model configurations.

7.2 Mapping Requirements to Software Architecture

Table-9: Matching requirements to software architecture

Requirements	Architectural Components
Users can register and receive an account pending admin approval.	User Manager
Administrators can review pending registrations and approve or deactivate user accounts.	User Manager
Users can log in with credentials and receive a signed JWT for secure session management.	User Manager
Users can reset a forgotten password via a one-time token sent to their registered email.	User Manager
Users can view a personalized dashboard showing activity statistics: saved projects, total smells detected, total refactoring operations, and recent activity.	Dashboard Manager

Users can generate unit tests automatically from Python source code using rule-based automated test generation.	Test Code Generator (Rule-Based: Engine)
Users can generate unit tests using LLM-based generation with project-aware dependency context.	Test Code Generator (LLM-Based: approach)
Users can detect test smells in existing or generated test suites using static rule-based analysis.	Test Smell Detector (Rule-Based: TEMPY Engine)
Users can detect test smells using LLM-based semantic analysis for deeper smell identification.	Test Smell Detector (LLM-Based: Ollama / HuggingFace)
Users can receive natural-language explanations for each detected smell to aid comprehension.	Test Smell Detector, Report Generator
Users can refactor detected test smells automatically using a single LLM agent.	Refactoring Engine
Users can apply a multi-agent refactoring strategy for validated, higher-quality refactoring outcomes.	Refactoring Engine (Multi-Agent LLM Workflow)
Users can save the state of their pipeline at any stage as a named, versioned snapshot.	Artifact Library Manager
Users can browse, inspect, and restore any previously saved pipeline version.	Artifact Library Manager
Users can delete individual project versions or entire projects.	Artifact Library Manager
Users can generate a comprehensive HTML pipeline report aggregating results from test generation, smell detection, and refactoring.	Report Generator, Test Code Generator, Test Smell Detector, Refactoring Engine
Users can save, retrieve, preview, and delete archived pipeline reports.	Report Generator, Artifact Library Manager

8. Deployment

This diagram helps to understand the deployment system of the project. To effectively deploy PyTestGenie, it's essential to have a clear and straight forward deployment architecture. Implementation. The deployment diagram of the PyTestGenie is shown below:

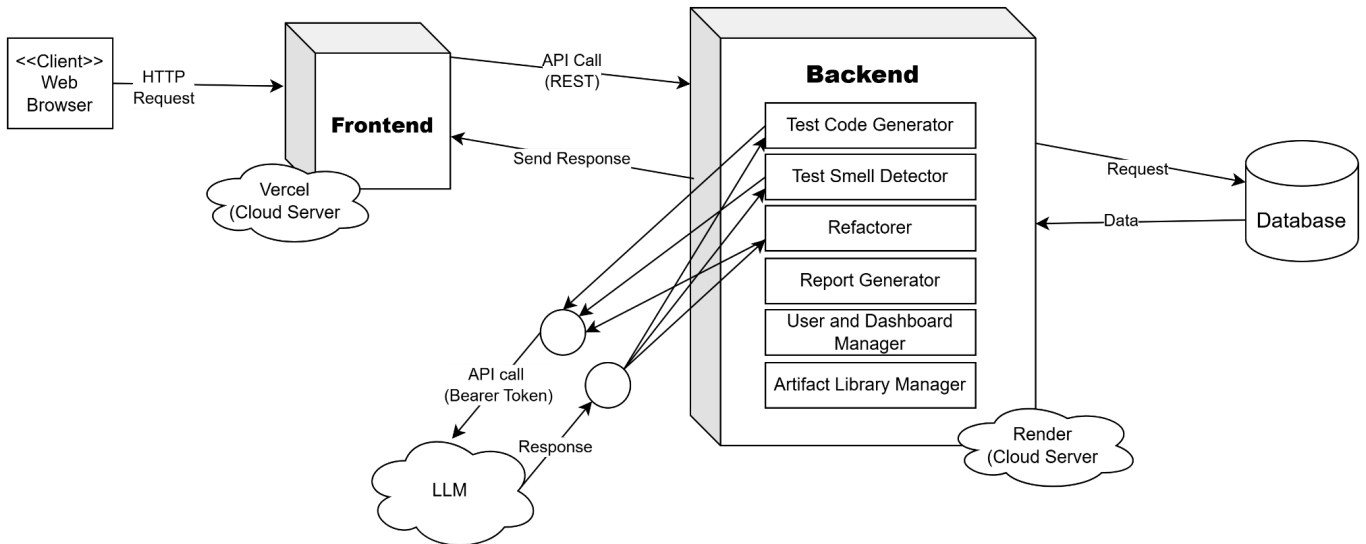


Figure 10: Deployment Diagram

9. Applied Methodology

A user brings in Python code, and the system helps them generate tests, find problems in those tests, and fix them. Each step builds on the previous one, but they can also be used independently. The whole process runs through a web interface, so no setup is required on the user's side. The methodology is not theoretical. Every step maps directly to a real action a user takes inside the tool.

9.1 Workflow

Here is the high level abstraction of the implemented PyTestGenie Project.

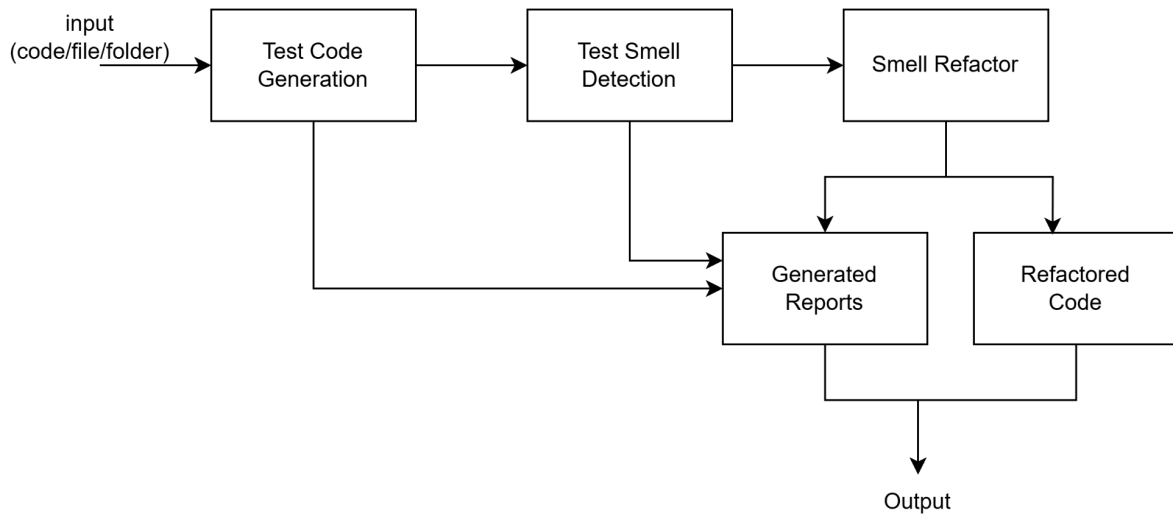


Figure 11: High level workflow

9.2 Input

PyTestGenie accepts input at different points in the workflow.

Source Code Input (for Test Generation)

The user provides the Python code they want to test. There are three ways to do this:

Method	How it works
Paste code	Type or paste code directly into the text area
Upload a file	Upload a single <code>.py</code> file from their computer
Upload a folder	Upload a complete Python project folder and pick one module

Only Python files (`.py`) are accepted. Any other file type is rejected immediately.

Test Code Input (for Smell Detection and Refactoring)

When using the Smell Detector or Refactorer directly (not through the Generator), the user provides test code. The options are:

Method	How it works
Paste code	Paste test code directly
Upload a file	Upload a single test <code>.py</code> file

Upload multiple files	Upload a folder of test files for batch analysis
GitHub URL	Paste a public repository URL and the system fetches the test files

Configuration Input

Beyond code, the user also makes a few choices that guide how the system runs:

- **Generation method:** Rule-based (Pynguin) or AI, plus the algorithm or model to use.
- **Detection method:** Rule-based or AI (LLM), plus the model provider and model name.
- **Smell type:** When refactoring, the user picks which specific problem to fix (or selects "All").
- **Agent mode:** For refactoring, either a single AI pass or a multi-step process with separate detect, evaluate, and fix stages.

9.3 Processing

Test Generation (Rule-Based)

1. The user's code is saved to a temporary folder on the server.
2. Pynguin runs against that folder using the selected algorithm (e.g., DYNAMOSA, RANDOM, MIO).
3. It explores the code by generating inputs and observing what happens.
4. The generated tests are collected and returned.
5. The temporary folder is deleted.

The user sees a live log of this process as it runs. Each log line is sent back over a server-sent event (SSE) stream so the page updates in real time.

Test Generation (LLM based)

1. The user's code is sent to the selected AI model along with a prompt asking it to write pytest-style unit tests.
2. The model returns test code as plain text.
3. The system forwards that code back to the user.

No server-side execution happens here. The AI writes the tests from reading the code alone.

Smell Detection (Rule-Based)

1. The test code is passed to an internal analyzer that parses it into a list of test methods.
2. Each method is checked against a set of rules, one per smell type.
3. Any rule that matches records the smell type, method name, and line number.
4. After all files are scanned, an HTML report is generated and the results are returned to the user.

The smell types the rule-based engine checks for:

Table-10: Smell types

Smell	What it looks for
Assertion Roulette	Multiple assertions with no explanatory message
Magic Number Test	Numeric literals used directly in assertions
Duplicate Assert	The exact same assertion written more than once
Exception Handling	Manual try/except blocks inside test methods
Conditional Test Logic	If, for, or while statements inside a test method
Missing Assertion	A test method with no assertion at all
Sleepy Test	Calls to <code>time.sleep()</code> inside a test
Undefined Test	A test that does nothing useful

Smell Detection (LLM-Based)

1. The test code and filename are sent to the selected AI model.
2. The prompt asks the model to identify any test smells and explain each one in plain language.
3. The model returns a structured list of smells.
4. The system parses and returns that list to the user.

Refactoring (Single Agent)

1. The system builds a prompt that explains the selected smell and what good code looks like.
2. The test code is included in the prompt.
3. The AI model rewrites the code to fix the smell.
4. The refactored code is returned.

Refactoring (Multi Agent)

The same goal, but split across three AI calls:

1. **Detect agent** : reads the code and decides whether the smell is present.
2. **Evaluate agent**: reviews the detection decision for accuracy.
3. **Refactored agent**: if the smell was confirmed, rewrite the code.

This takes longer but produces more reliable results on tricky cases.

Authentication and Session Handling

Every user action that involves saved data (projects, versions, reports) requires a valid login token. The token is a JWT (JSON Web Token) issued at login. It is stored in the browser and sent with every request. Tokens expire after 30 minutes of inactivity. When a token expires, the user is redirected to the login page automatically.

Data Storage

All persistent data is stored in a MongoDB database:

Collection	What it stores
users	Accounts, passwords (hashed), approval status
projects	Project records linked to a user
project_versions	Saved snapshots of work at each pipeline stage
pipeline_reports	Saved HTML reports

9.4 Output

Generated Test Code

The primary output of the Test Generator is a Python test file. It follows pytest or unittest conventions depending on the method used. The user can:

- Copy it to their clipboard.
- Download it as a .py file.
- Pass it directly to the Smell Detector.

Smell Detection Results

The Smell Detector produces two forms of output:

On-screen results table shows each detected smell with:

- The smell type (e.g., Assertion Roulette)
- The test method name where it was found
- The line number(s)
- For LLM detection: a plain-language explanation

Refactored Code

The Refactorer returns a rewritten version of the test code. The original test logic is preserved. Only the structural problems are fixed. The user can copy or download the refactored code, or save it as a project version.

Project Versions

At any stage, the user can save a snapshot. A saved version stores:

Field	Contents
Label	A name given by the user (e.g., "v1_ Initial Version")
Step type	What stage this is from (source, tests generated, smells detected, refactored)
Source code	The original Python code
Test code	The generated or provided test code
Smell results	Detected smells and counts
Refactored code	The cleaned-up test code

Pipeline Report

After completing a full workflow, the user can generate a pipeline report. This is an HTML document that summarizes the entire session in one place:

- The source code used
- The tests that were generated
- The smells that were found
- The refactored code

Reports are saved to the Report Archive and can be viewed or re-opened any time.

9.5 Code Implementation Overview

PyTestGenie is a web-based platform that allows developers to generate unit tests, detect quality issues in those tests, and refactor them directly from a browser. The system is built using a Python Flask backend and a React frontend. The following sections briefly describe how each major backend component is implemented and how they work together.

9.5.1 app_unified.py

This file serves as the main entry point of the backend and assembles the entire application during server startup. It follows Flask's Application Factory Pattern using the `create_app(config_name)` function. Multiple Blueprints are registered under `/api/*` routes to modularize features such as authentication, test generation, smell detection, refactoring, reporting, and project management. A global CORS policy is applied using an `after_request` hook to enable secure communication with the React frontend.

9.5.2 services/auth_service.py and middleware/auth.py (Security and Authentication)

These modules implement user registration, login, password reset, and access control. Passwords are securely hashed using `bcrypt`, and JWT tokens (HS256, 30-minute expiry) are generated upon successful login. New users are stored with `is_approved = False` until an administrator approves them. The password reset workflow includes

token generation, email dispatch, verification, and secure password updating. The `@token_required` decorator in `middleware/auth.py` protects restricted routes by verifying JWT tokens, validating account status, and attaching the authenticated user to `request.current_user`. Invalid or expired tokens immediately return HTTP 401 responses.

9.5.3 modules/test_generator/ (Test Code Generation Engine)

This component automatically generates unit tests for uploaded or pasted Python files using either AI-based or rule-based methods. In AI mode, `ai_generator.py` resolves local dependencies, constructs a structured prompt, queries models such as GPT-OSS 20B (HuggingFace) or Llama 3.2 (Ollama), removes markdown formatting from responses, validates syntax using `ast.parse()`, and returns a structured result. In rule-based mode, `pynguin_generator.py` executes the Pynguin CLI as a subprocess with a selected search algorithm (DYNAMOSA, MIO, MOSA). Real-time execution logs are streamed to the browser using Server-Sent Events (SSE).

9.5.4 modules/smell_detector/ (Test Smell Detection Engine)

This component analyzes test files to detect structural and behavioral test smells. `TestSmellAnalyzer` acts as the main entry point, parsing files using an AST-based `PythonParser` with a fallback mechanism. The Detector applies ten predefined TEMPY rules to identify issues such as Conditional Test Logic, Exception Handling misuse, Redundant Print statements, Sleepy Tests, Unknown Tests, Verbose Tests, Paradigms Blend, Verifying in Setup, Non-Functional Statements, and Undefined Tests. In AI mode, `llm_smell_service.py` prompts an LLM with a twelve-smell catalogue and parses structured JSON responses. Optionally, `GeminiClient` generates short natural-language explanations and fix suggestions using Gemini 1.5 Flash.

9.5.5 routes/refactoring.py (Refactoring Engine)

This module automatically refactors smelly test code using LLMs. The `get_llm()` function initializes either Ollama-based local models or HuggingFace cloud models. In single-agent mode, a focused prompt containing the smell definition and test code is

sent to the model for direct rewriting. In multi-agent mode, a four-stage pipeline is executed: Detect Agent confirms smell presence, Eval-Detect Agent validates detection, Refactor Agent rewrites the code, and Eval-Refactor Agent verifies correctness and cleanliness. Seven smell types are supported, and multiple LLM backends are available, including Llama 3.2, Phi4, Mistral, CodeLlama, Mistral 7B, and GPT-OSS 20B.

9.5.6 models/ (Data Persistence Layer)

This layer manages all MongoDB interactions using the Repository Pattern. UserModel handles user creation, lookup, approval, and deactivation with unique indexes for usernames and emails. ProjectModel manages projects and versioned snapshots, where each version captures a complete pipeline state with an auto-incremented version number. PipelineReportModel stores rendered HTML reports with metadata and timestamps. All database queries are strictly filtered by user_id to ensure per-user data isolation.

9.5.7 config/settings.py (Configuration Management)

This module centralizes environment-specific settings such as database URLs, API tokens, and secret keys. The base Config class loads values from a .env file using python-dotenv. Key variables include MONGODB_URL, SECRET_KEY, HF_TOKEN, GEMINI_API_KEY, LLAMA_API_URL, and CORS_ORIGINS. Separate DevelopmentConfig and ProductionConfig classes extend the base configuration with environment-specific overrides. The configuration dictionary is selected during application startup, ensuring no credentials are hard-coded.

9.5.8 modules/smell_detector/report_generator.py (Report Generation)

This component generates styled HTML reports from smell detection results. Using a Builder Pattern, the ReportGenerator constructs the report incrementally in memory and writes it to disk in a final step. It creates summary headers, per-file tables, detailed smell rows, and optionally includes AI-generated explanations. The final output is written to

report/log.html or report/log_ai.html. All CSS is embedded inline, making the report fully self-contained and viewable offline.

9.5.9 routes/pipeline_report.py and routes/projects.py (Artifacts Library)

This component manages persistent storage of reports and versioned project snapshots. The pipeline report archive provides endpoints for saving, listing, retrieving, rendering, and deleting reports. The project history module allows users to create named projects, save pipeline versions, retrieve specific versions, and delete them safely. Version numbers are auto-incremented, and deleting the final remaining version of a project is prevented to avoid accidental data loss. All operations are restricted to the authenticated user's data.

9.5.10 routes/dashboard.py (Dashboard Manager)

The dashboard provides a personalized overview of user activity. It retrieves statistics such as total saved project versions, total reports, aggregated smell counts, and total refactoring sessions using MongoDB queries and aggregation pipelines. It also generates a unified activity feed by merging and sorting recent project and report records. Additionally, users can securely change their password through a protected endpoint that verifies the current password using bcrypt before saving a new hashed version.

10. User Manual and User Interface

10.1 Introduction

PyTestGenie is a web-based automated testing assistant designed for Python developers. It brings together three core capabilities in one place:

- **Test Code Generation:** automatically write unit tests for your Python code using rule-based tools or llm-based approach.
- **Test Smell Detection :** scan your test files for common quality problems (called test smells) and get a detailed report.

- **Test Code Refactoring:** automatically clean up and improve test code that has known quality issues.

This manual walks through every page and feature of the application so that any user, regardless of technical background can get started quickly.

10.2 System Requirements

- A modern web browser (Google Chrome, Firefox, Microsoft Edge, or Safari).
- An internet connection.
- A PyTestGenie account (provided or registered through the application).

No installation is required on the user's computer.

10.3 Accessing PyTestGenie

Open your browser and navigate to the PyTestGenie URL provided.

The first page you see will be the Login page.

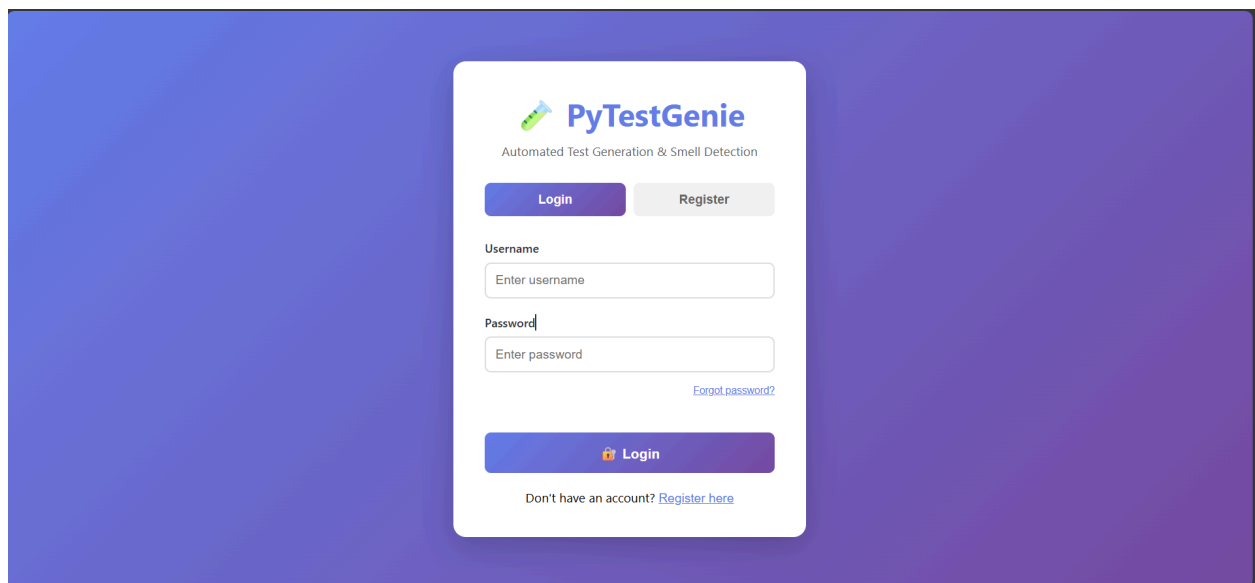


Figure 12: Login Page

10.4 Registering a New Account

If you do not already have an account, you can create one from the Login page.

Steps:

1. On the Login page, click "Register here"
2. Fill in the following fields:
 - Username: choose a unique name (no spaces).
 - Email Address: enter a valid email.
 - Password: choose a strong password(at least 6 character).
3. Click Register.
4. A message will appear confirming that your account has been created.

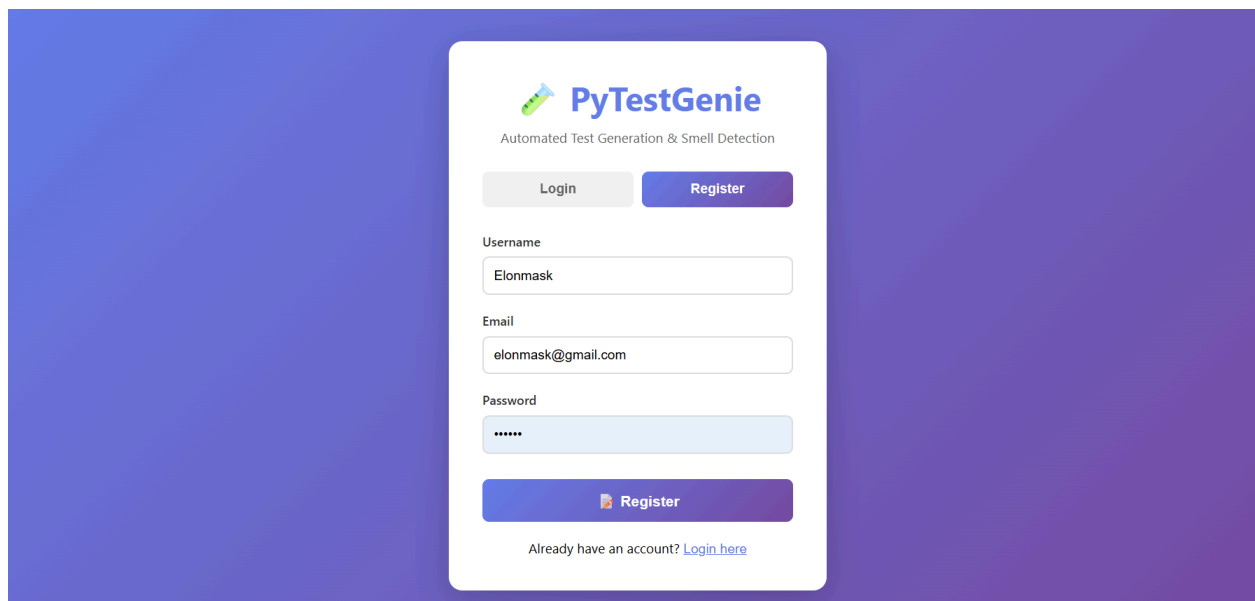
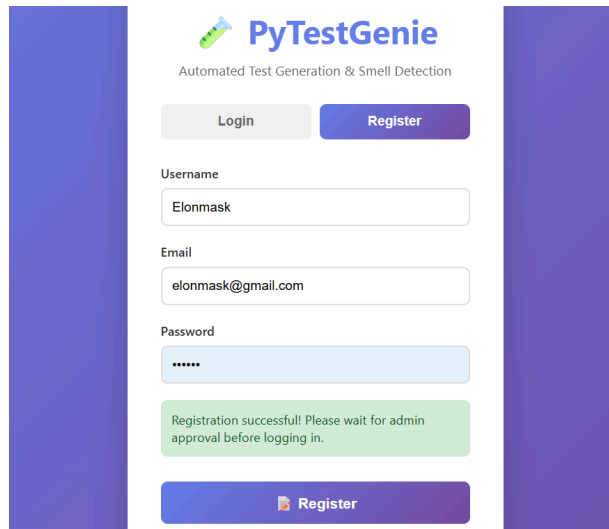
The image shows a registration form for 'PyTestGenie' on a purple gradient background. The form is a white card with rounded corners. At the top, it features the 'PyTestGenie' logo (a green and blue icon) and the tagline 'Automated Test Generation & Smell Detection'. Below this are two buttons: a grey 'Login' button and a blue 'Register' button. The form contains three input fields: 'Username' with the text 'Elonmask', 'Email' with 'elonmask@gmail.com', and 'Password' with six dots. A large blue 'Register' button is positioned below the password field. At the bottom of the card, there is a link that says 'Already have an account? [Login here](#)'.

Figure 13: Registration form

Note: Newly registered accounts are not active immediately. An administrator must approve your account before you can log in. You will see a message saying your account is pending approval after you register.



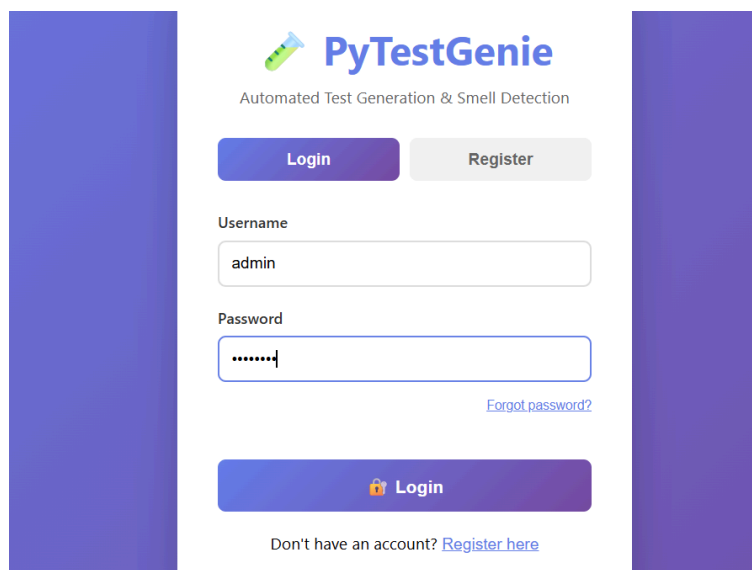
The image shows the PyTestGenie registration page. At the top, there is a logo with a test tube icon and the text "PyTestGenie" followed by the tagline "Automated Test Generation & Smell Detection". Below this are two buttons: "Login" and "Register". The "Register" button is highlighted. Underneath, there are three input fields: "Username" with the value "Elonmask", "Email" with the value "elonmask@gmail.com", and "Password" with masked characters "*****". A green message box states: "Registration successful! Please wait for admin approval before logging in." At the bottom, there is a "Register" button with a lock icon.

Figure 14: "Pending approval" message shown after successful registration

10.5 Logging In

Steps:

1. On the Login page, enter your Username and Password.
2. Click Login.
3. If your credentials are correct and your account has been approved, you will be taken to the application.



The image shows the PyTestGenie login page. At the top, there is a logo with a test tube icon and the text "PyTestGenie" followed by the tagline "Automated Test Generation & Smell Detection". Below this are two buttons: "Login" and "Register". The "Login" button is highlighted. Underneath, there are two input fields: "Username" with the value "admin" and "Password" with masked characters "*****". A link "Forgot password?" is visible below the password field. At the bottom, there is a "Login" button with a lock icon. Below the login button, there is a link "Don't have an account? Register here".

Figure 15: Login page with credential entered

Message	Cause
Invalid credentials	Username or password is wrong.
Account not approved	Your account is registered but waiting for admin approval.
Token is missing	Your session expired, log in again.

10.6 The Application Layout

After logging in, you will see the main application interface. It has three main areas:

1. **Header (top bar)** : shows the PyTestGenie logo, the application name, your logged-in username, and a Logout button.
2. **Navigation bar**: a row of tabs to switch between the different tools.
3. **Main content area**: the active tool or page is displayed here.

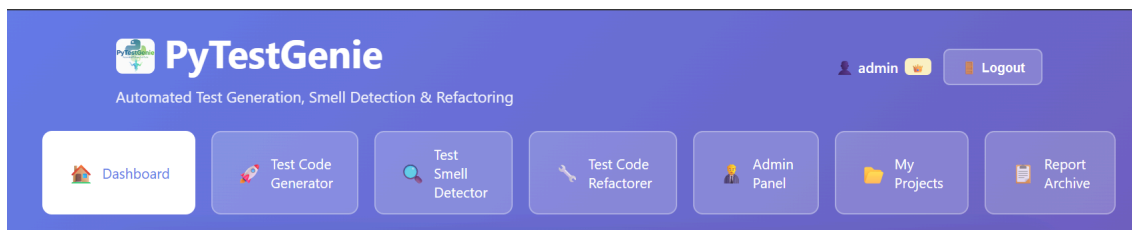


Figure 16: Application window after login

Navigation tabs available:

Tab	What it does
Dashboard	Overview of your activity and statistics
Test Code Generator	Generate unit tests for your Python code
Test Smell Detector	Scan test files for quality issues
Test Code Refactorer	Fix quality issues in test code automatically

My Projects	View and manage saved versions of your work
Report Archive	Access previously saved HTML reports
Admin Panel	(Admin accounts only) Manage users

10.7 Dashboard

The Dashboard is the first page you see after logging in. It gives you a quick summary of everything you have done on the platform.

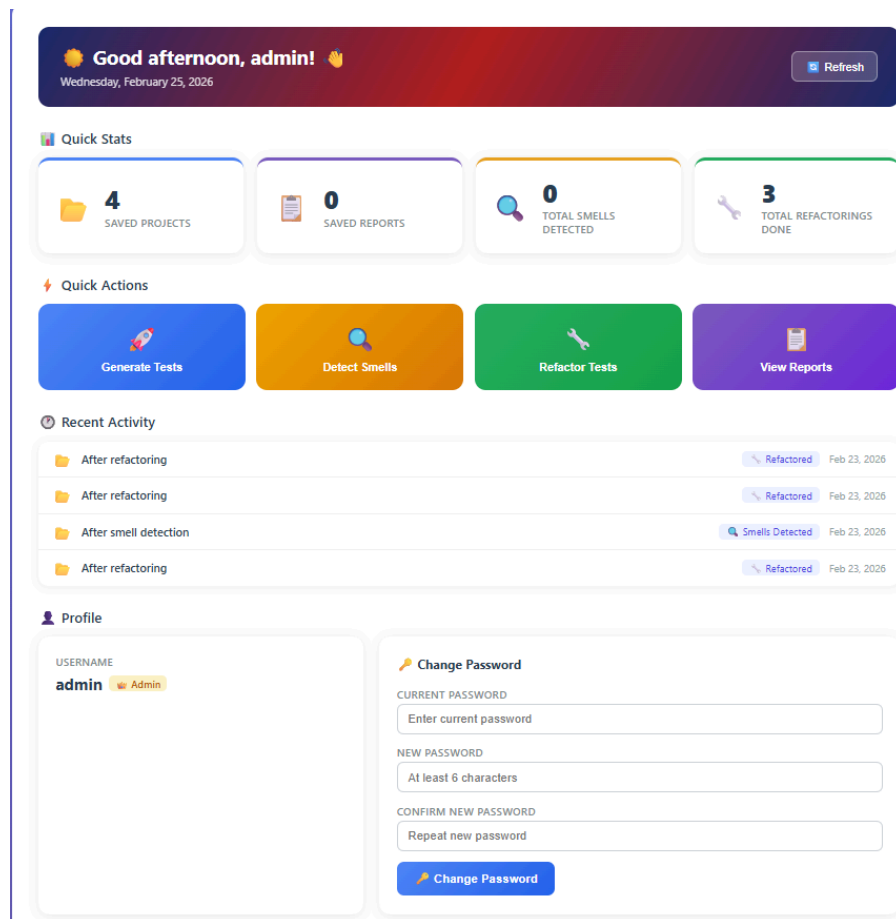


Figure 17: Dashboard

10.7.1 Statistics Cards

Four cards at the top of the dashboard show:

Card	What it counts
Saved Projects	Total number of project versions you have saved
Saved Reports	Total number of HTML reports in your archive
Smells Detected	Total number of test smells found across all your work
Refactorings Done	Total number of times you have saved a refactored version

Clicking on a card will take you directly to the relevant section.

10.7.2 Recent Activity

Below the stat cards, a list of your 10 most recent actions is displayed, showing what you worked on and when.

10.7.3 Change Password

At the bottom of the Dashboard there is a Change Password section.

Steps:

1. Enter your current password.
2. Enter a new password.
3. Re-enter the new password to confirm it.
4. Click Change Password.

Figure 18: Change Password Window

10.8 Test Code Generator

This tool generates unit tests for any Python function or class you provide.

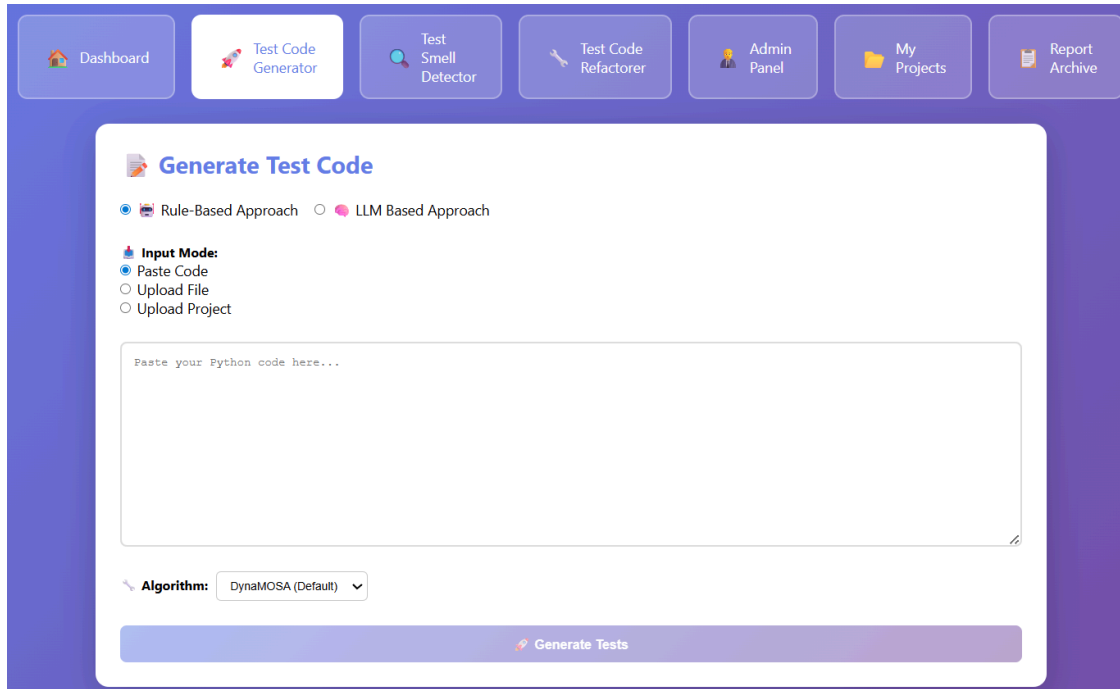
The screenshot shows the 'Test Code Generator' page of a web application. At the top, there is a navigation bar with seven buttons: 'Dashboard', 'Test Code Generator' (highlighted), 'Test Smell Detector', 'Test Code Refactorer', 'Admin Panel', 'My Projects', and 'Report Archive'. The main content area is titled 'Generate Test Code'. It features two radio buttons for 'Rule-Based Approach' (selected) and 'LLM Based Approach'. Below this, there is an 'Input Mode' section with three radio buttons: 'Paste Code' (selected), 'Upload File', and 'Upload Project'. A large text area is provided for pasting Python code, with the placeholder text 'Paste your Python code here...'. Below the text area is an 'Algorithm' dropdown menu currently set to 'DynaMOSA (Default)'. At the bottom of the form is a prominent 'Generate Tests' button.

Figure 19: Test Code Generator page

10.8.1 Providing Your Code

You have three ways to provide the code you want to test:

Option 1 (Paste Code): Click the *Paste Code* tab and type or paste your Python code directly into the text area.

Option 2 (Upload a File): Click the *Upload File* tab, then click Choose File and select a `.py` file from your computer. The file contents will be loaded automatically.

Option 3 (Upload a Project Folder): Click the *Upload Project* tab to upload an entire Python project folder. After uploading, you can select which module inside the project you want to generate tests for.

10.8.2 Choosing a Generation Method

Once you have provided your code, choose how you want tests to be generated:

Rule-Based

- Select the Rule-Based option.
- Choose an algorithm from the dropdown (e.g., *DYNAMOSA*, *RANDOM*, *MIO*).
- This method uses automated search to generate tests based on code structure.

Generate Test Code

☒ Rule-Based Approach
 ☐ LLM Based Approach

Input Mode:

☐ Paste Code
☒ Upload File
☐ Upload Project

✓ prime_num.py

```

def is_prime_for_else(number):
    if number <= 1:
        return False
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            break
    else: # This block runs if no 'break' occurred
        return True
    return False
  
```

Algorithm: DynaMOSA (Default) ▼

- DynaMOSA (Default)
- MIO
- MOSA

Figure 20: Rule based test case generation page

LLM-Based

- Select the LLM based approach option.
- Choose a model provider (*Ollama* for local models, *HuggingFace* for cloud models).
- Select the specific model (e.g., *Llama 3.2*, *Mistral 7B*, *GPT-OSS 20B*).
- The AI reads your code and writes tests the way a developer would.

Generate Test Code

☐  Rule-Based Approach ☒  LLM Based Approach

Input Mode:

☐ Paste Code
☒ Upload File
☐ Upload Project

 Choose Python File

✓ prime_num.py

```
def is_prime_for_else(number):  
    if number <= 1:  
        return False  
    for i in range(2, int(number**0.5) + 1):  
        if number % i == 0:  
            break  
    else: # This block runs if no 'break' occurred  
        return True  
    return False
```

 AI Model:

GPT-OSS 20B (HuggingFace) ▼

GPT-OSS 20B (HuggingFace)

Llama 3.2 (Local)

 Generate Tests

Figure 21: LLM based test case generation page

10.8.3 Generating Tests

Click the Generate Tests button to start.

- For the Rule-Based method, a live progress log will appear showing what is happening in real time.
- For the Llm based method, a loading indicator will appear while the AI processes your code.

🔥 Generate Tests

📋 Generation Logs

```

Using algorithm: DYNAMOSA
[17:04:21] INFO Starting Pynguin client with master-worker client.py:40
architecture
[17:04:27] INFO Received result for task master.py:133
test_gen_1772017461.9910407: 0
INFO Stopping master-worker system client.py:44
INFO Master-worker system stopped client.py:46
        
```

✅ Generated Test Code

```

# Test cases automatically generated by Pynguin (https://www.pynguin.eu).
# Please check them before you use them.
import user_code as module_0

def test_case_0():
    float_0 = 0.5
    var_0 = module_0.is_prime_for_else(float_0)
    assert var_0 is False

def test_case_1():
    float_0 = 2746.0
    var_0 = module_0.is_prime_for_else(float_0)
    assert var_0 is False

def test_case_2():
    float_0 = 2745.190802292324
    var_0 = module_0.is_prime_for_else(float_0)
    assert var_0 is True
    var_1 = module_0.is_prime_for_else(var_0)
    assert var_1 is False
        
```

📄 Copy

⬇️ Download

🔍 Detect Test Smells

💾 Save to Project

Detect by: ☒ Rule-Based ☐ LLM-Based

Figure 22: Rule based response for test code generation

🤖 AI Model:

Llama 3.2 (Local) ▼

🔥 Generate Tests

✅ Generated Test Code

```

import pytest
from typing import Callable, TypeVar

T = TypeVar('T')

def is_prime_for_else(number: int) -> bool:
    if number <= 1:
        return False
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            break
    else:
        return True
    return False

@pytest.fixture
def test_number():
    yield [1, 2, 4, 6, 8, 9]
    yield [10]

@pytest.mark.parametrize("test_num", test_number)
def test_is_prime(test_num: int) -> None:
    """Test the function with valid inputs"""
    for num in test_num:
        assert is_prime_for_else(num)

@pytest.mark.parametrize("test_num", test_number)
def test_is_not_prime_negative(test_num: int) -> None:
    """Test the function with negative numbers"""
    for num in test_num:
        
```

📄 Copy

⬇️ Download

🔍 Detect Test Smells

💾 Save to Project

Detect by: ☒ Rule-Based ☐ LLM-Based

Figure 23: LLM based response for test code generation

10.8.4 Viewing the Generated Tests

When generation completes, the generated test code appears in the output panel on the right side of the screen.

- You can copy the test code using the Copy button.
- You can download it as a `.py` file.

10.8.5 Running Smell Detection on Generated Tests

After tests are generated, a *Detect Smells* button appears. This lets you immediately check the generated tests for quality issues without leaving the page.

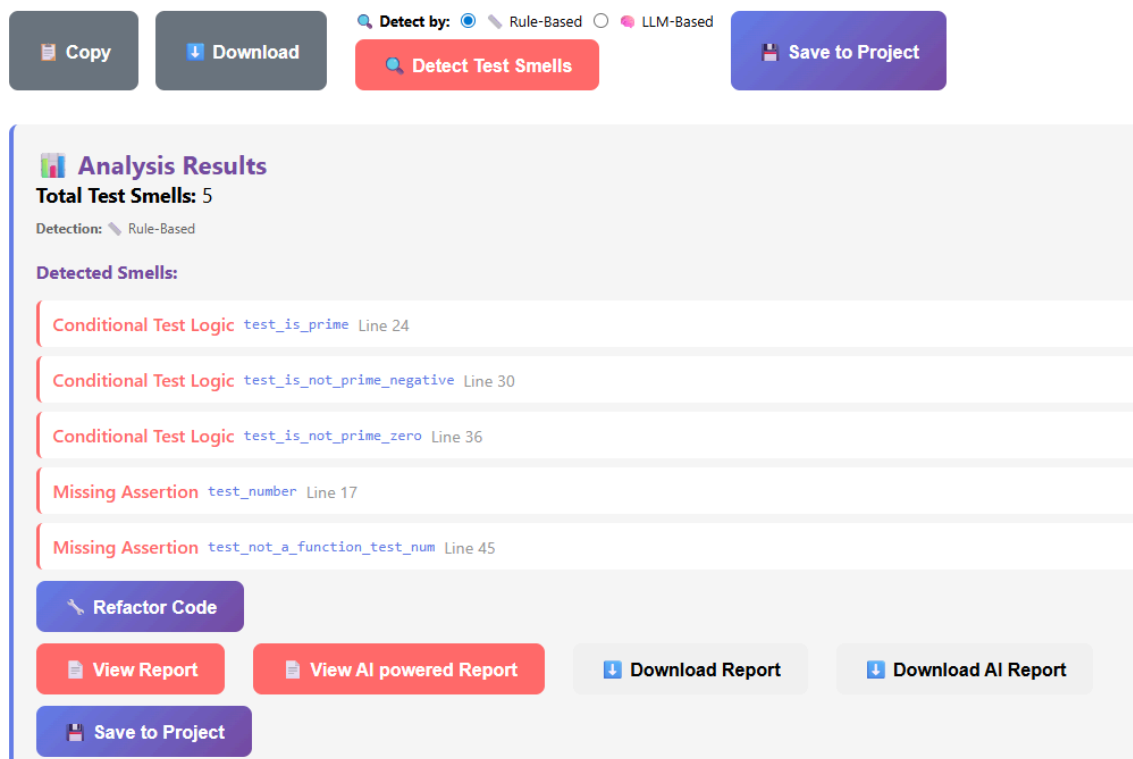


Figure 24: Detected smell viewer

10.8.6 Saving a Version

To save your current work (source code + generated tests) as a project version, click *Save Version*. A dialog will ask for a project name, version label, and step type.

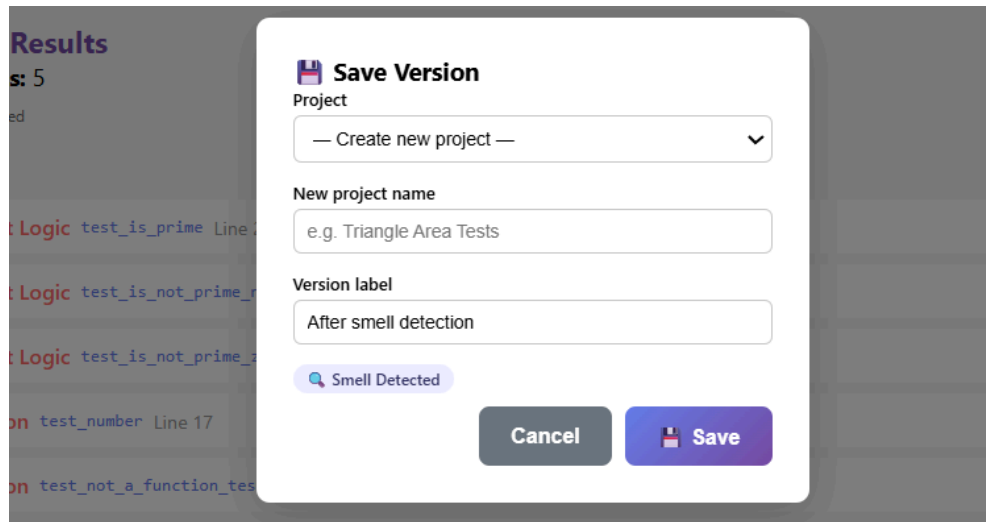


Figure 25:Save version window

10.8.7 Generating a Pipeline Report

After completing generation and optionally smell detection or refactoring, click *Generate Report* button to create a full HTML pipeline report summarizing the entire workflow.

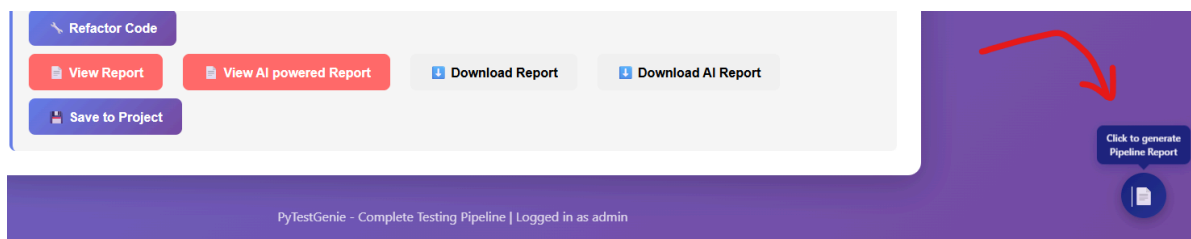


Figure 26:Button for report generation

10.9 Test Smell Detector

This tool scans Python test files for common quality problems known as test smells.

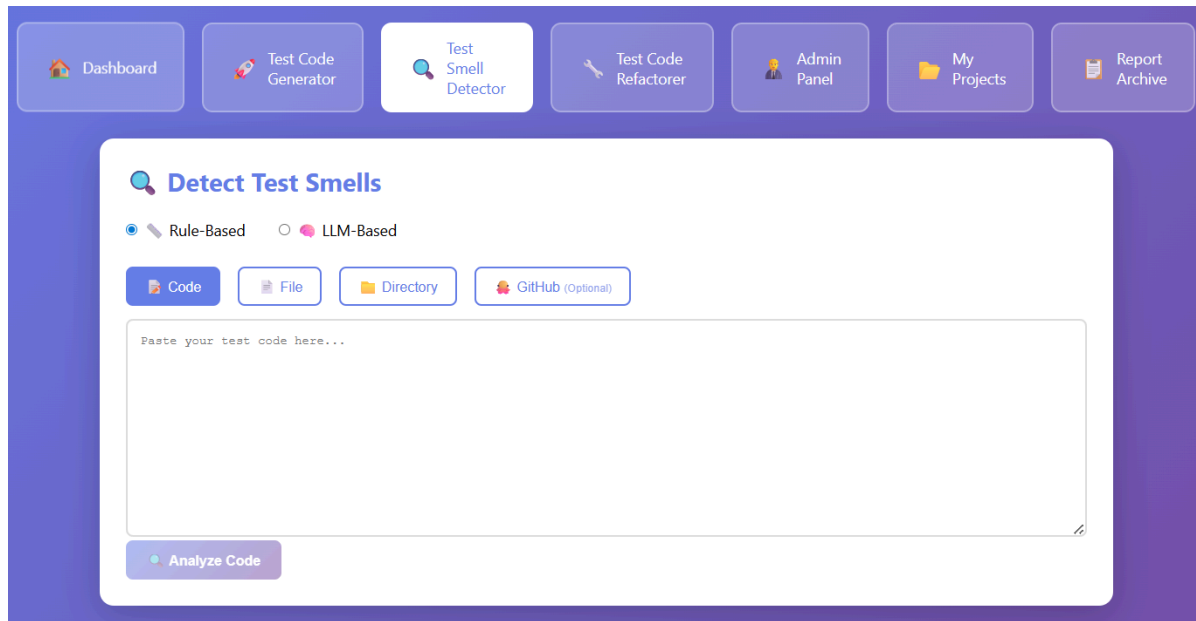


Figure 27:Test Smell Detector page

10.9.1 Input Modes

Paste Code: Type or paste your test code directly into the text area and click *Analyze Code*.

Upload a Single File: Click on the *Upload File* button, choose a `.py` test file, and click *Analyze File*.

Upload Multiple Files (Directory): Click on the *Upload Directory* button to select multiple files at once and analyze them all together.

Analyze a GitHub Repository: Click on the *GitHub URL* tab, paste a public GitHub repository URL, and click *Analyze Repository*.

10.9.2 Choosing a Detection Method

Rule Based Detection: Uses built-in rules to detect specific patterns

LLM Based Detection: Choose the model provider and model name from the dropdowns that appear.

The screenshot shows the 'Detect Test Smells' application interface. At the top, there's a title 'Detect Test Smells' with a magnifying glass icon. Below it, two radio buttons are present: 'Rule-Based' (unselected) and 'LLM-Based' (selected). Under the 'LLM-Based' section, there are two dropdown menus: 'Provider' set to 'HuggingFace API' and 'Model' set to 'Mistral 7B Instruct v0.2'. A blue tooltip message says 'LLM detects smells with line numbers & method names — may take a moment.' Below these are three buttons: 'Code' (highlighted in blue), 'File', and 'Directory'. The 'Directory' button is open, showing a list of models: 'Mistral 7B Instruct v0.2' and 'GPT-OSS 20B'. At the bottom, there's a text input field with the placeholder 'Paste your test code here...'.

Figure 28: LLM based test detector options

10.9.3 Viewing Detection Results

After analysis, the results panel shows:

- Total smells detected .
- A table listing each smell with:
 - Smell type (e.g., Assertion Roulette, Magic Number Test)
 - Method name where it was found
 - Line number in the file

Analysis Results

Total Test Smells: 11

Detected Smells:

- Conditional Test Logic `test_wrap_around_behavior` Line 45
- Conditional Test Logic `test_fifo_order` Line 73
- Conditional Test Logic `test_fifo_order` Line 75
- Conditional Test Logic `test_size_never_exceeds_max` Line 98
- Conditional Test Logic `test_multiple_wrap_around_cycles` Line 105
- Conditional Test Logic `test_indices_within_bounds` Line 136
- Conditional Test Logic `test_head_tail_reset_after_empty` Line 145
- Conditional Test Logic `test_head_tail_reset_after_empty` Line 147
- Conditional Test Logic `test_repeated_operations` Line 187
- Missing Assertion `test_enqueue_non_int_raises_type_error` Line 90
- Missing Assertion `test_invalid_initialization` Line 114

Detection: Rule-Based

[View Report](#)

[View AI powered Report](#)

[Download Report](#)

[Download AI Report](#)

[Refactor Code](#)

[Save to Project](#)

Figure 29:Smell detection results panel showing a table of detected smells

For LLM-based detection, each smell also includes a plain-language explanation.

[Detect Test Smells](#)

Analysis Results

Total Test Smells: 25

Detected Smells:

Assertion Roulette `test_queue_initialization`

The test method contains multiple assertions without descriptive messages. For example, the assert statements on `'queue_small.size'`, `'queue_small.max'`, `'queue_small.head'`, `'queue_small.tail'`, and `'isinstance(queue_small.data, array.array)'` are all grouped together without clear separation or explanation. This makes it harder to identify which assertion failed if the test fails. The developer should split these assertions into individual test cases or add descriptive messages to clarify the expected behavior.

Assertion Roulette `test_enqueue_until_full`

The test method contains multiple assertions without descriptive messages. For instance, the assertions on `'queue_small.enqueue(10)'`, `'queue_small.enqueue(20)'`, `'queue_small.enqueue(30)'`, and `'queue_small.full()'` are all grouped together. If the test fails, it is unclear which specific enqueue operation or condition caused the failure. The developer should add descriptive messages or split these assertions into separate test cases.

Figure 30 :LLM-based detection result with explanation text visible

10.9.5 Refactoring from the Detector

After detection, a *Refactor Code* button appears. Clicking it opens the inline refactoring panel where you can fix the detected issues without switching pages.

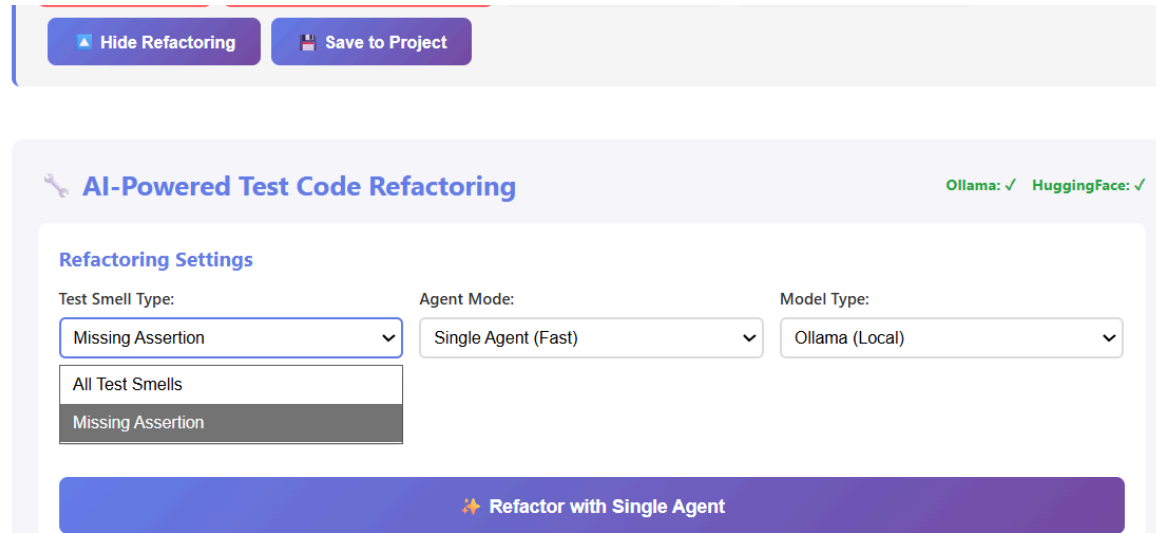


Figure 31 :Refactoring panel after test smell detection

10.10 Test Code Refactorer

This tool takes test code with known quality problems and rewrites it to be cleaner and more maintainable using an AI model.

AI-Powered Test Code Refactorer

Refactor your test code to remove test smells using AI agents

 Ollama: ✓ Available  HuggingFace: ✓ Configured

 Paste Code

 Upload File

Test Code:

Paste your test code here...

Refactoring Settings

Test Smell Type:

All Test Smells

Agent Mode:

Single Agent (Fast)

Model Type:

Ollama (Local)

Model Name:

Llama 3.2

 Refactor with Single Agent

 Clear All

Figure 32 :Refactorer Page

10.10.1 Providing Test Code

Paste Code: type or paste the test code directly.

Upload File: click *Choose File*, select a `.py` test file, and the contents will be loaded automatically.

10.10.2 Selecting a Smell to Fix

From the *Select Smell* dropdown, choose the specific quality problem you want to fix:

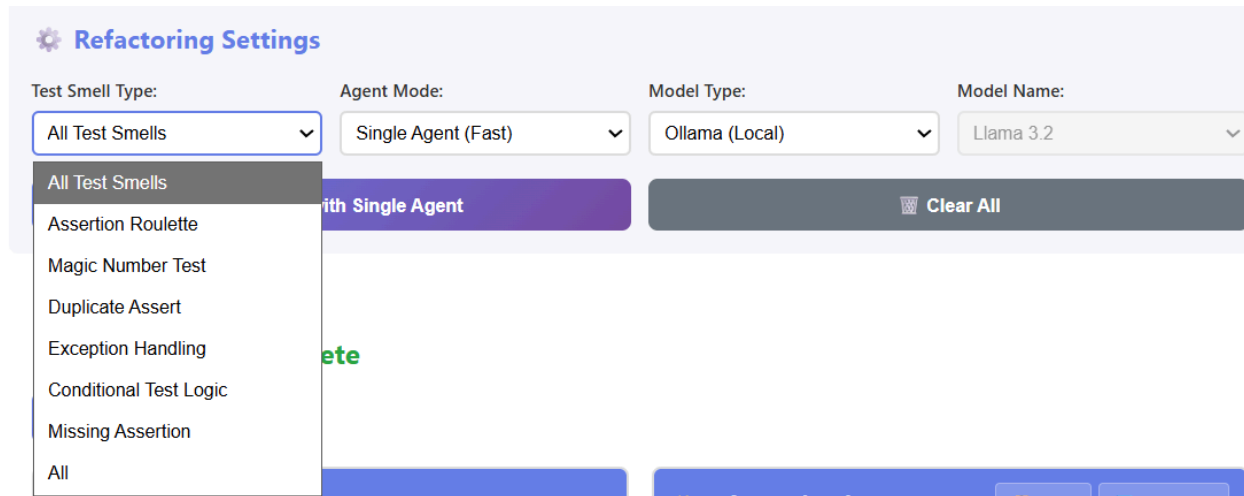


Figure 33 :Test Smell Type Dropdown menu

10.10.3 Choosing an AI Model

Select the model provider:

- **Ollama** : uses a locally running AI model (e.g., Llama 3.2).
- **HuggingFace**: uses cloud-based models (e.g., Mistral 7B, GPT-OSS 20B).
Requires internet.

Then select the specific model from the second dropdown.

10.10.4 Choosing an Agent Mode

- Single Agent
- Multi Agent

10.10.5 Running the Refactoring

Click *Refactor Code*. The system will send your code to the AI model and wait for the refactored version.

10.10.6 Viewing the Refactored Code

When done, the refactored test code appears in the output panel. You can:

- Copy the code to use elsewhere.
- Save Version to keep this as a project snapshot.
- Generate Report to create a full pipeline report.

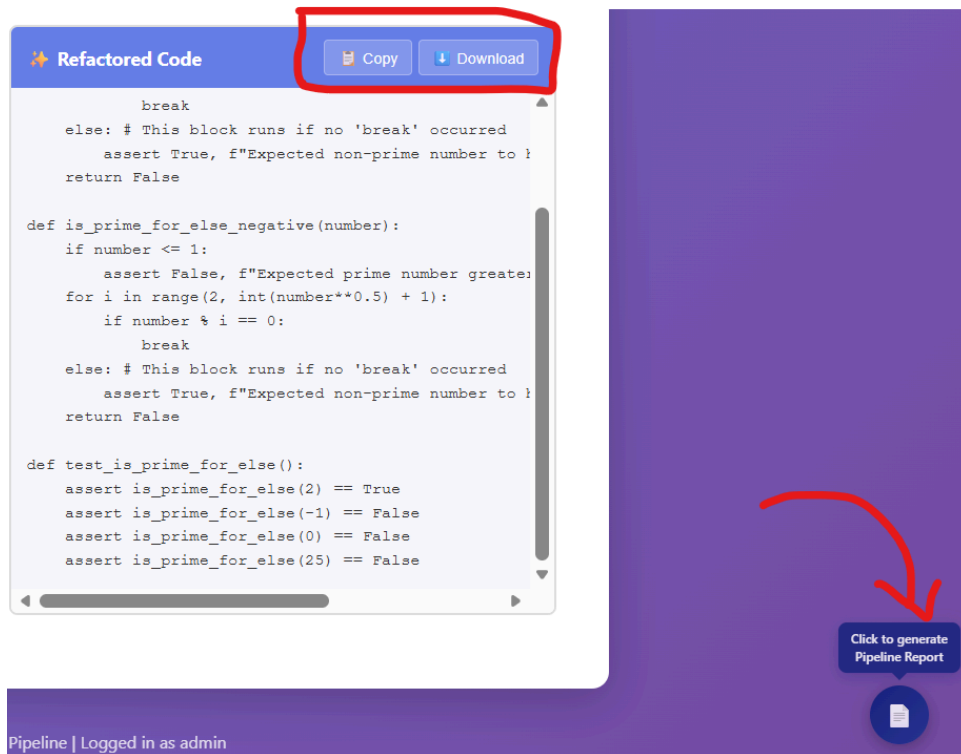


Figure 34 :Refactored output page (Single Agent Mode)

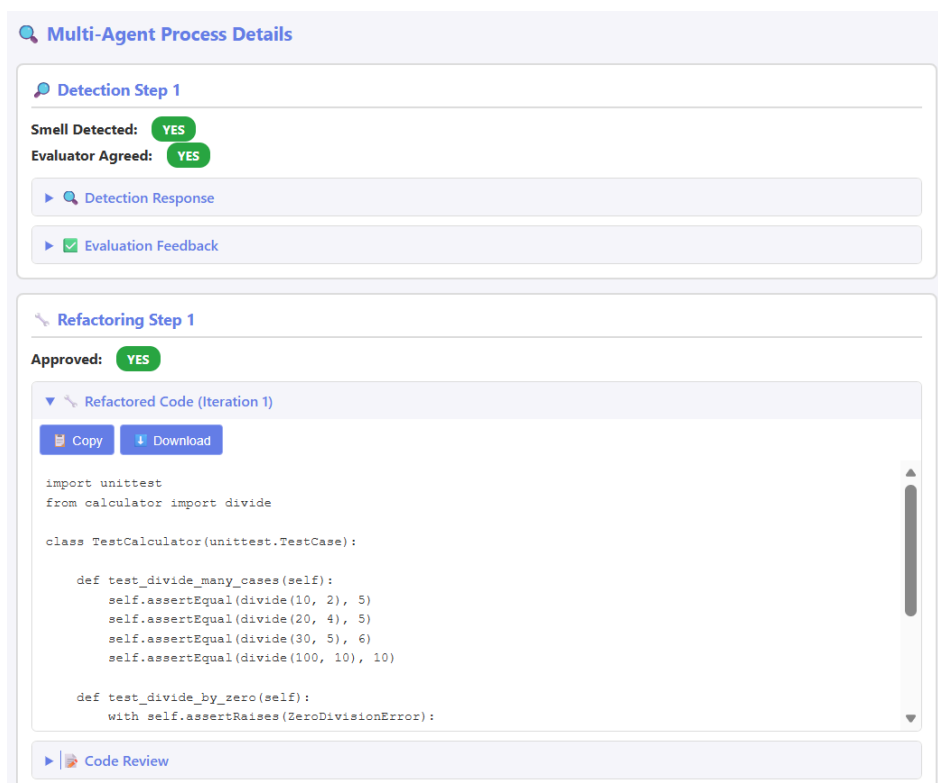


Figure 35 :Refactored output page (Multi Agent Mode)

10.11 My Projects (Artifacts Library)

The *My Projects* page stores a complete history of your work, organized by project. Each version you save across any tool appears here.

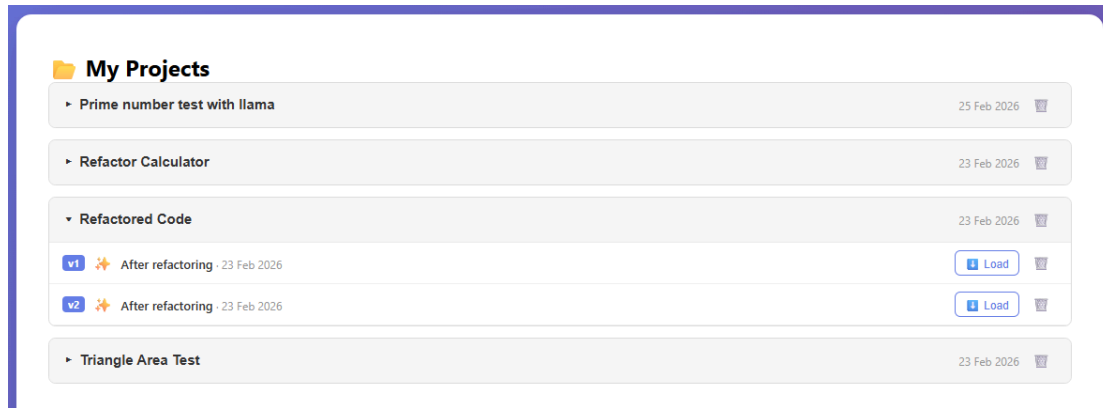


Figure 36 :My Projects page showing a list of projects

10.11.1 Deleting a Version or Project

- Click *Delete* next to a version to remove that.
- Click *Delete Project* to remove an entire project and all its versions.

A confirmation prompt will appear before anything is permanently deleted.

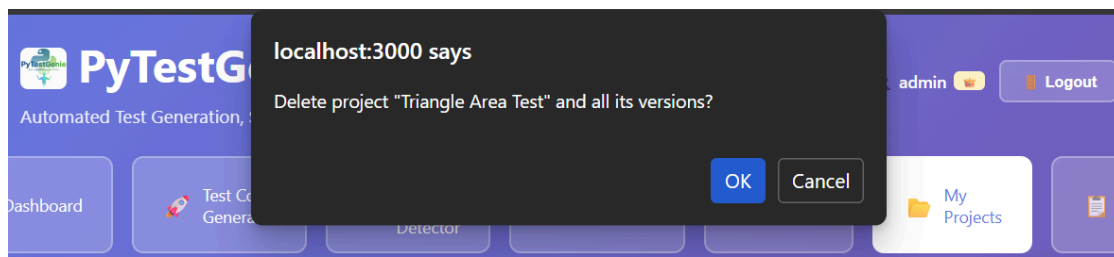


Figure 37 :Delete confirmation dialog

10.12 Report Archive

The *Report Archive* stores all HTML pipeline reports you have generated and saved. Reports give you a complete picture of a full workflow (tests generated, smells detected, refactored).



Figure 38 :Report Archive page showing a list of saved reports with titles and dates

10.12.1 Viewing a Report

Click *View* on any report to open the full HTML report in a panel or a new browser tab.

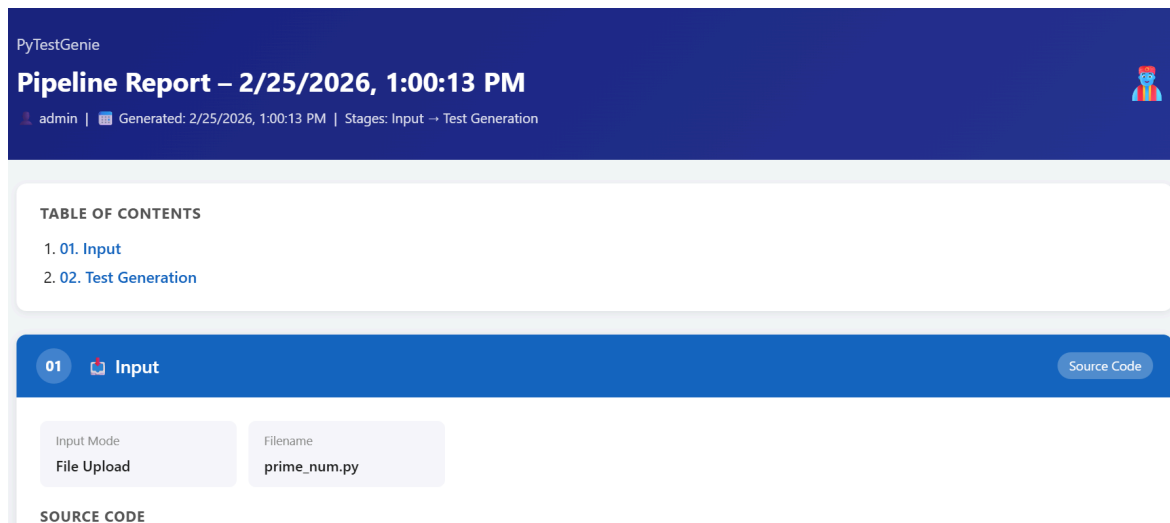


Figure 39 :Report displayed showing all sections of the pipeline summary

10.12.2 Deleting a Report

Click *Delete* next to a report and confirm. The report will be permanently removed from your archive.

10.13 Admin Panel (Admin accounts only)

The Admin Panel is only visible to users with administrator privileges. It provides tools to manage all user accounts on the platform.

Admin Panel

Manage users and approvals

 Pending Approvals (2)

 All Users (5)

admin ★ Admin admin@pytestgenie.com ✓ Approved ● Active	
amit amitroy50773@gmail.com ✓ Approved ● Active	Deactivate Delete
mos bsse1314@iit.du.ac.bd ✓ Approved ● Active	Deactivate Delete
Solo Sikoa testhello1@yopmail.com ⌚ Pending ● Active	Deactivate Delete
Elonmask elonmask@gmail.com ⌚ Pending ● Active	Deactivate Delete

 Refresh

Figure 40 :Admin Panel page showing the user management table

10.13.1 Viewing All Users

The main table shows every registered user with their: Username, Email address, Account status (Approved / Pending / Inactive). Role (Admin / Regular user)

10.13.2 Approving a Pending User

New registrations appear with a *Pending* status. To allow them to log in:

1. Find the user in the list.
2. Click *Approve*.
3. The status changes to *Approved* and the user can now log in.

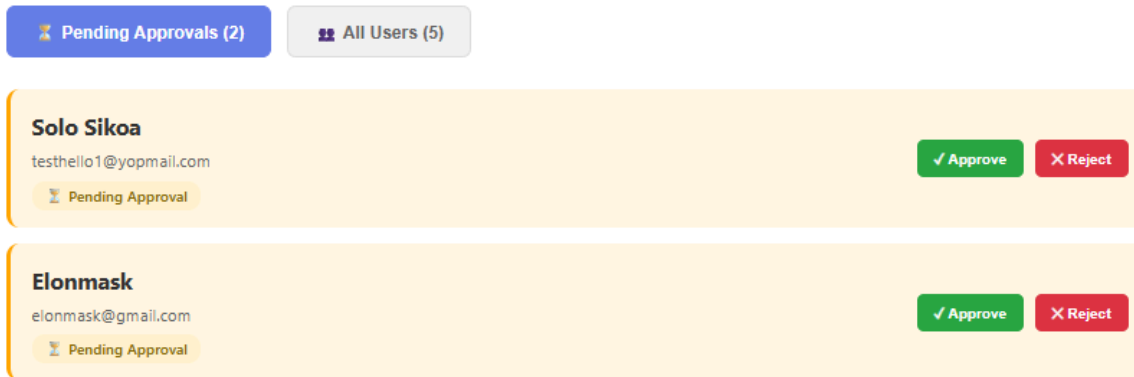


Figure 41 : A pending user state

10.13.3 Deactivating or Deleting a User

- **Deactivate:** the user cannot log in but their data is kept.
- **Delete:** the user is permanently removed.

10.14 Logging Out

To end your session, click *Logout* in the top-right corner of the header. You will be taken back to the Login page and your session token will be cleared.

10.15 Frequently Asked Questions (Assumption)

Q: My registration was successful but I cannot log in. Why?

Ans: Your account is waiting for an administrator to approve it. Contact your admin and ask them to approve your account in the Admin Panel.

Q: The test generation is taking a very long time.

Ans: Rule-based generation (Pynguin) can take a few minutes depending on the complexity of the code. The live log will keep updating so you can see progress. For AI generation, response time depends on the selected model and server load.

Q: The AI-based refactoring returned code that looks the same.

Ans: If the AI does not detect the selected smell in the code, it returns the original code unchanged. Try selecting a different smell type or use "All" to check for all smells at once.

Q: I accidentally deleted a project version. Can I recover it?

Ans: No. Deletions are permanent. Always save important versions before deleting older ones.

Q: What is the difference between Single Agent and Multi Agent refactoring?

Ans: Single Agent uses one AI pass to detect and fix the problem, it is faster. Multi Agent uses three separate AI steps (detect, evaluate, fix), it takes more time.

Q: Can I upload a non-Python file?

Ans: No. The system only accepts `.py` (Python) files. Uploading any other file type (`.txt`, `.java`, `.js`, etc.) will be rejected with an error message.

11. Test Plan

This chapter presents a high-level description of testing goals and a set of test cases designed to validate the functionality, accuracy, and usability of *PyTestGenie*. You can find test report here: [Test Report Link](#)

11.1 High-Level Description of Testing Goals

The primary testing goals for *PyTestGenie* are as follows:

- To validate that *PyTestGenie* meets its functional requirements, including test code generation, smell detection, refactoring, and artifact management.
- To ensure that automated test generation produces syntactically correct and semantically meaningful unit tests using both rule-based algorithms and LLM-based methods.
- To verify that test smell detection accurately identifies design and maintainability issues in generated or existing test suites using both rule-based and LLM-based approaches.

- To ensure that multi-agent LLM-based refactoring correctly transforms smelly tests into high-quality tests while preserving functionality.
- To validate that the artifacts library accurately stores uploaded code, generated test suites, refactored tests, and reports, and allows easy retrieval.
- To test the performance and scalability of PyTestGenie when multiple users generate and analyze tests concurrently.
- To confirm the usability of the dashboard and interfaces for code upload, viewing results, and accessing reports.

11.2 Test Cases

Below is a set of representative test cases for *PyTestGenie*.

Module 1: Authentication

Test Case 1: User Registration

Test Scenario: A new user creates an account on PyTestGenie for the first time. Steps:

1. Fill in a username, email address, and password on the registration form.
2. Submit the registration request.
3. Check that the account has been saved in the system.

Expected Outcome:

- The system accepts the registration and responds with a success message.
- The user's password is stored in a secure, unreadable format (not as plain text).
- The account is marked as "pending" until an admin approves it.

Result: Pass. A new account is created securely and waits for admin approval before it can be used.

Test Case 2: User Registration (Missing Fields)

Test Scenario: A user tries to register but leaves out required information. Steps:

1. Submit the registration form with only a username, leaving email and password blank.

Expected Outcome:

- The system rejects the request immediately.
- A clear error message is shown asking the user to fill in all required fields.

Result: Pass. The system does not allow incomplete sign-up forms to go through.

Test Case 3: User Registration (Duplicate Username)

Test Scenario: A user tries to register using a username that someone else already has.

Steps:

1. Register successfully with the username "testuser01".
2. Try to register again using the exact same username.

Expected Outcome:

- The second attempt is rejected.
- The system informs the user that the username or email is already taken.

Result: Pass. The system prevents two accounts from sharing the same username.

Test Case 4: User Login with Correct Credentials

Test Scenario: A user who has been approved by an admin logs in successfully. Steps:

1. Make sure the account exists and has been approved.
2. Enter the correct username and password on the login form.

Expected Outcome:

- The system grants access and returns a login token.
- The user can now use the application.

Result: Pass. Approved users can log in and receive a session token without any issues.

Test Case 5: User Login (Wrong Password)

Test Scenario: A user enters the wrong password when trying to log in. Steps:

1. Enter a valid username but an incorrect password.

Expected Outcome:

- The system denies access.
- An error message is displayed.

Result: Pass. Wrong passwords are rejected and no access is granted.

Test Case 6: User Login (Unapproved Account)

Test Scenario: A newly registered user tries to log in before an admin approves their account. Steps:

1. Register a new account (it is pending approval by default).
2. Try to log in right away without waiting for admin approval.

Expected Outcome:

- The system denies access.
- The user is informed that their account has not been approved yet.

Result: Pass. Users cannot access the system until an admin explicitly approves their account.

Test Case 7: View Own Profile

Test Scenario: A logged-in user views their own account information. Steps:

1. Log in and obtain a session token.
2. Open the profile page using that token.

Expected Outcome:

- The system returns the user's username, email, and role.
- The password is never shown in the response.

Result: Pass. Users can view their profile and sensitive data like passwords is never exposed.

Test Case 8: Access Restricted Page Without Logging In

Test Scenario: Someone tries to open a page that requires login without being logged in. Steps:

1. Try to access the profile page without providing any login token.

Expected Outcome:

- The system blocks the request.
- A message is returned saying the token is missing.

Result: Pass. Pages that require login are fully protected from unauthenticated access.

Module 2: Test Code Generation

Test Case 9: Rule-Based Test Generation (rule-based)

Test Scenario: A user generates automated unit tests using the rule-based method.

Steps:

1. Paste or upload a Python function.
2. Select the rule-based method and choose the "DYNAMOSA" algorithm.
3. Start the test generation process.

Expected Outcome:

- The system accepts the request and begins generating tests in the background.
- A live progress log is shown to the user as generation happens.
- The process finishes with a completion signal.

Result: Pass. The system generates tests automatically and shows live progress to the user.

Test Case 10: Rule-Based Generation (No Code Submitted)

Test Scenario: A user tries to start test generation without providing any code. Steps:

1. Submit the test generation form without any code in the input field.

Expected Outcome:

- The system immediately rejects the request.
- A message is shown asking the user to provide code first.

Result: Pass. The system will not start generating tests if there is no code to work with.

Test Case 11: LLM-based Test Generation

Test Scenario: A user generates tests using an AI language model instead of rules.
Steps:

1. Paste a Python function into the input field.
2. Select the AI method and choose a model (e.g., GPT-OSS).
3. Start generation.

Expected Outcome:

- The system returns a complete test file written by the AI.
- The generated tests are valid Python code with proper structure.
- The response shows which model was used.

Result: Pass. The AI model generates readable, valid tests for the given function.

Module 3: File Upload

Test Case 12: Upload a Python File

Test Scenario: A user uploads a Python source file to use with the tools. Steps:

1. Click the upload button and choose a .py file from their computer.
2. Submit the upload.

Expected Outcome:

- The file is accepted and its contents are made available.
- The user can immediately use the uploaded code for test generation or smell detection.

Result: Pass. Python files upload successfully and are ready to use right away.

Test Case 13: Upload a Non-Python File (Rejected)

Test Scenario: A user accidentally tries to upload a file that is not a Python file. Steps:

1. Choose a .txt file and try to upload it.

Expected Outcome:

- The system rejects the file.
- A message tells the user that only .py files are accepted.

Result: Pass. The system only accepts Python files and clearly explains this when another type is submitted.

Module 4: Test Smell Detection

Test Case 14: Detect Smells in an Uploaded Test File (Rule-Based)

Test Scenario: A user uploads a test file and checks it for common test quality issues using built-in rules. Steps:

1. Write or prepare a test file that contains known problems (e.g., many assertions with no explanation messages).
2. Upload the file and choose the rule-based detection option.

Expected Outcome:

- The system identifies the problems and lists each one with the method name and line number where it was found.
- A summary report is generated.

Result: Pass. Test quality issues are detected and clearly reported with their locations in the code.

Test Case 15: Detect Smells from Pasted Code (Rule-Based)

Test Scenario: A user pastes test code directly into the tool instead of uploading a file. Steps:

1. Paste test code into the text input area.
2. Run the rule-based smell detection.

Expected Outcome:

- The system analyzes the pasted code and returns a list of any detected issues.

Result: Pass. The detection tool works equally well with pasted code, no file upload needed.

Test Case 16: Detect Smells Using AI (LLM-Based)

Test Scenario: A user detects test smells using an AI model that also provides explanations. Steps:

1. Paste the test code.
2. Select the AI-based detection option with a local model (e.g., Llama 3.2).

Expected Outcome:

- The system returns a list of issues found by the AI.
- Each issue includes a plain-language explanation of why it is a problem.

Result: Pass. The AI model detects issues and explains them in a way that is easy to understand.

Test Case 17: Upload Wrong File Type to Smell Detector (Rejected)

Test Scenario: A user tries to upload a non-Python file to the smell detection tool. Steps:

1. Select a .txt file and upload it to the smell detection form.

Expected Outcome:

- The system rejects the file.
- An error message tells the user to upload a .py file.

Result: Pass. The smell detector enforces file type rules just like the other upload tools.

Module 5: Smell Refactoring

Test Case 18: Fix Assertion Roulette

Test Scenario: A user cleans up a test that has many assertions without any explanation of what each one checks. Steps:

1. Paste or upload a test file that has Assertion Roulette.
2. Select "Assertion Roulette" as the smell to fix and choose a model.
3. Run the refactoring.

Expected Outcome:

- The refactored code has a short, clear message added to every assertion.
- The actual test logic stays exactly the same; only the messages are added.

Result: Pass. The tool adds meaningful messages to each assertion without changing what the test does.

Test Case 19: Fix Magic Number Test

Test Scenario: A user cleans up a test that uses raw numbers in assertions without naming them. Steps:

1. Paste test code that has unnamed numbers in assertions (e.g., `assertEqual(result, 42)`).
2. Select "Magic Number Test" as the smell and run refactoring.

Expected Outcome:

- The refactored code replaces the raw numbers with clearly named constants (e.g., `EXPECTED_RESULT = 42`).

Result: Pass. Raw numbers are given descriptive names, making the test much easier to read and maintain.

Module 6: Report Generator

Test Case 20: Save a Report

Test Scenario: A logged-in user saves the result of their analysis as an HTML report for later viewing. Steps:

1. Log in.
2. After running a detection or generation, save the result as a report with a title.

Expected Outcome:

- The report is saved successfully.
- The system returns a confirmation with the report ID and title.

Result: Pass. Reports are saved and linked to the user's account for future reference.

Test Case 21: View List of Saved Reports

Test Scenario: A user wants to see all the reports they have previously saved. Steps:

1. Log in.

2. Open the reports section to see the archive.

Expected Outcome:

- All previously saved reports are listed with their titles and dates.
- The full report content is not loaded for every item (only summaries are shown in the list).

Result: Pass. The archive loads quickly and shows only the necessary information in the list view.

Test Case 22: Delete a Saved Report

Test Scenario: A user removes an old report they no longer need. Steps:

1. Find the report they want to delete from their archive.
2. Click delete and confirm.
3. Try to open the deleted report again.

Expected Outcome:

- The report is removed.
- Trying to access it afterwards shows a "not found" message.

Result: Pass. Deleted reports are permanently removed and cannot be accessed again.

Module 7: Dashboard Management

Test Case 23: View Dashboard Overview

Test Scenario: A logged-in user checks their personal dashboard to see a summary of all their activity. Steps:

1. Log in.
2. Navigate to the dashboard.

Expected Outcome:

- The dashboard shows four key numbers: total saved projects, total reports, total smells detected, and total refactorings done.
- A short list of recent activity (last 10 actions) is also shown.

Result: Pass. The dashboard accurately reflects all the user's work at a glance.

Test Case 24: Dashboard Blocked for Non-Logged-In Users

Test Scenario: Someone tries to open the dashboard without being logged in.

Steps:

1. Try to access the dashboard page without providing a login token.

Expected Outcome:

- Access is denied.
- The system redirects to the login page.

Result: Pass. The dashboard is fully protected and cannot be viewed without logging in first.

Module 8: Artifacts Library

Test Case 25: Create a New Project

Test Scenario: A user starts a new project in the Artifacts Library to organize their work.

Steps:

1. Log in.
2. Go to the projects section and create a new project with a name and description.

Expected Outcome:

- The project is created and appears in the user's project list immediately.

Result: Pass. New projects are created instantly and ready to hold versions and artifacts.

Test Case 26: Save a Project Version

Test Scenario: A user saves the current state of their work as a named version under a project. Steps:

1. Open an existing project.
2. Save a version with a label (e.g., "v1.0 Prime Num Code Tested") and the current code or results.

Expected Outcome:

- The version is saved and linked to the project.
- The system confirms the save with the version ID and label.

Result: Pass. Versions are saved correctly and remain accessible for comparison and review later.

Test Case 27: Browse Project Versions

Test Scenario: A user reviews all the saved versions of a project to track their progress.
Steps:

1. Open a project that has multiple saved versions.
2. View the version history.

Expected Outcome:

- All versions are listed with their labels and type (e.g., source code, generated tests, smell results, refactored).

Result: Pass. All saved snapshots are displayed in an organized way, showing the full history of the project.

Test Case 28: Delete a Project

Test Scenario: A user removes a project they no longer need along with all its saved versions. Steps:

1. Select the project to delete.
2. Confirm the deletion.

Expected Outcome:

- The project and all its versions are removed.
- Trying to access it afterwards shows a "not found" message.

Result: Pass. Projects and all their associated data are fully removed when deleted.

Module 9: Admin Module

Test Case 29: Admin Views All Registered Users

Test Scenario: An admin checks the list of everyone who has signed up on the platform.
Steps:

1. Log in with an admin account.
2. Open the user management section.

Expected Outcome:

- A full list of all registered users is shown, including those still waiting for approval.
- Non-admin users cannot access this page .

Result: Pass. Only admins can view and manage the full user list.

Test Case 30: Admin Approves a Pending User

Test Scenario: An admin reviews a new registration and approves the user so they can log in. Steps:

1. A new user registers (their account is pending by default).
2. The admin logs in and finds the pending user.
3. The admin approves the account.
4. The user tries to log in after being approved.

Expected Outcome:

- The approval action succeeds.
- The user's account status changes to approved.
- The user can now log in successfully and use the application.

Result: Pass. The approval process works.

Conclusion

PyTestGenie provides a comprehensive and integrated framework for automated unit test generation, test smell detection, and test code refactoring for Python programs. By combining traditional rule-based techniques with advanced LLM-based approaches, the system allows developers to generate high-quality, executable tests efficiently. The integration of role-based multi-agent LLM workflows further enhances the system's ability to detect complex test smells and apply intelligent refactorings, bridging gaps that purely rule-based systems may miss.

The ArtifactsLibraryManager ensures that all uploaded code, generated tests, refactored tests, and reports are stored securely, providing users with a centralized library for managing and analyzing their work. The preliminary test plan demonstrates that the system meets functional requirements, handles edge cases, and provides robust performance under concurrent operations.

In conclusion, PyTestGenie significantly reduces manual testing effort, improves code quality, and empowers developers and researchers with a unified platform for both automated and AI-assisted Python test management, ultimately enhancing software reliability and maintainability.

References

- [1] S. Lukasczyk and G. Fraser, “Pynguin: Automated unit test generation for python,” in Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ser. ICSE ’22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 168–172, isbn: 9781450392235. doi: 10.1145/3510454.3516829. [Online]. Available: <https://doi.org/10.1145/3510454.3516829>
- [2] Github link: <https://github.com/AmitRoy01/PyTestGenie>
- [3] R Melo, P Simões, R Gheyi, M d’Amorim, M Ribeiro, G Soares, E Almeida, E Soares, “[Agentic LMs: Hunting Down Test Smells](#)”arXiv preprint arXiv:2504.07277, 2025.
- [4] D Fernandes, I Machado, R Maciel, “[TEMPY: Test smell detector for Python](#)”. Proceedings of the XXXVI Brazilian Symposium on Software Engineering, 2022
- [5]PyNose: A Test Smell Detector For Python, Tongjie Wang, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, Iftekhar Ahmed. Available link: <https://arxiv.org/abs/2108.04639>
- [6]Test Report link: <https://docs.google.com/spreadsheets/d/1GP7tkZNs16QFCEbwNEAmkXcyXPRHd8DSz8NtWpcguAI/edit?usp=sharing>